

DOSSIER PROJET

Titre Professionnel Développeur Web et Web Mobile (DWWM)

VITE & GOURMAND

Application Web de Traiteur Événementiel

URL : <https://vite-gourmand.fr>

Candidat : Jamesy MUKUNA MUKENKETAYI Email : yvesnet9@gmail.com — Téléphone : +32 465 18 81 99

Formation : Graduate Développeur web full stack 2023-2029 — STUDI (DIGITAL CAMPUS LIVE), Paris

Session : Novembre – Décembre 2026

Entreprise réalisatrice : FastDev (entreprise de services numériques fictive) **Client :** Vite & Gourmand – Julie et José (Bordeaux)

Sommaire

1 Introduction.....	3
1.1 Contexte du projet.....	3
1.2 Objectifs du projet.....	3
1.3 Durée et organisation.....	3
2 Analyse des besoins.....	3
2.1 Identification des acteurs.....	3
2.2 Cas d'usage principaux.....	4
3 Modélisation des données.....	8
3.1 Modèle Conceptuel de Données (MCD) – PostgreSQL.....	8
3.2 Structure de la table Users.....	9
3.3 Normalisation.....	9
3.4 Schéma de base de données complet (PostgreSQL).....	9
3.5 Modèle NoSQL (MongoDB).....	12
3.6 Diagrammes de séquence.....	13
4 Architecture technique.....	14
Conception : maquettes.....	14
Charte graphique.....	15
4.1 Vue d'ensemble.....	15
4.2 Choix techniques justifiés.....	16
4.3 Architecture frontend.....	17
4.4 Architecture backend.....	17
5 Sécurité.....	27
5.1 Authentification (Laravel Sanctum).....	27
5.2 Validation des données (FormRequests).....	27
5.3 Contrôle d'accès (middleware de rôle + Politiques).....	27
5.4 Gestion des secrets.....	27
5.5 Protection du transport.....	27
5.6 Rate limiting.....	28
5.7 CORS.....	28
6 Conformité RGPD.....	29
6.1 Droits implémentés.....	29
6.2 Gestion du consentement.....	29
6.3 Sécurité des données.....	29
7 Notifications par e-mail (Resend).....	29
8 Tests.....	30
8.1 Tests automatisés (PHPUnit).....	30
8.2 Exécution des tests.....	30
8.3 Couverture fonctionnelle.....	30
8.4 Apport des tests : détection d'une faille d'autorisation.....	31
9 Déploiement en production.....	31
9.1 Infrastructure.....	31
9.2 Configuration Nginx.....	31
9.3 Processus de déploiement.....	32
9.4 Mise en production.....	32
9.5 Configuration PHP-FPM.....	33
9.6 Configuration PostgreSQL.....	33
9.7 Optimisations de performance.....	33
9.8 Monitoring et sauvegardes.....	33
10 Réalisation.....	34

10.1 Fonctionnalités développées.....	34
10.2 Accessibilité (RGAA).....	35
10.3 Captures d'écran des interfaces.....	35
11 Difficultés rencontrées et solutions.....	39
11.1 Accès SSH au VPS.....	39
11.2 Configuration de l'API frontend (CRA vs Vite).....	40
11.3 Routing Nginx pour l'API.....	40
11.4 Extension MongoDB en production.....	40
11.5 Faille d'autorisation détectée par les tests.....	41
12 Bilan du projet.....	41
12.1 Objectifs atteints.....	41
12.2 Compétences développées.....	41
12.3 Métriques du projet.....	42
12.4 Perspectives d'évolution.....	42
12.5 Bilan personnel.....	42
13 Conclusion.....	42
14 Annexes.....	43
14.1 Annexe A : comptes de démonstration.....	43
14.2 Annexe B : liens utiles.....	43
14.3 Annexe C : technologies et versions.....	43
14.4 Annexe D : glossaire.....	44
14.5 Annexe E : maquettes (wireframes).....	45
14.6 Annexe F : charte graphique.....	49

1 Introduction

1.1 Contexte du projet

Dans le cadre de ma formation au Titre Professionnel Développeur Web et Web Mobile (DWWM) chez STUDI, j'ai réalisé un projet complet de développement d'une application web pour un service de traiteur événementiel nommé « Vite & Gourmand ».

Ce projet répond à un besoin réel de digitalisation d'une entreprise de traiteur qui souhaite offrir à ses clients la possibilité de consulter et commander des menus en ligne, tout en facilitant la gestion opérationnelle pour l'équipe.

Le projet a d'abord été livré en production, puis enrichi de manière itérative : ajout d'une base NoSQL pour les statistiques, mise en place des e-mails transactionnels, affinage des règles métier (statuts de commande, frais de livraison, remises, caution matériel), renforcement de la sécurité et mise en accessibilité (RGAA).

1.2 Objectifs du projet

Les objectifs principaux de ce projet sont :

- Permettre aux clients de consulter les menus disponibles et de passer des commandes en ligne
- Faciliter la gestion des commandes pour l'équipe (validation, suivi, livraison)
- Offrir un outil d'administration complet pour gérer les menus, plats, allergènes et utilisateurs
- Produire des **statistiques d'activité** (chiffre d'affaires par menu) au moyen d'une base **NoSQL (MongoDB)**
- Informer les clients par **e-mails transactionnels** (confirmation, suivi)
- Assurer la sécurité des données et la conformité RGPD
- Garantir un niveau d'**accessibilité (RGAA)** sur les pages principales
- Déployer l'application en production sur un serveur sécurisé avec HTTPS

1.3 Durée et organisation

Durée du développement initial : 12 jours (du 6 au 18 février 2026), suivie de plusieurs itérations d'amélioration.

Méthode : développement itératif en suivant les fonctionnalités F01 à F12, puis cycles d'amélioration continue.

Résultat : application fonctionnelle déployée en production sur <https://vite-gourmand.fr>.

2 Analyse des besoins

2.1 Identification des acteurs

J'ai identifié quatre types d'acteurs principaux avec des besoins et des permissions différents.

2.1.1 Visiteur (non connecté)

- Consultation des menus disponibles
- Inscription pour créer un compte
- Accès aux pages RGPD (politique de confidentialité, mentions légales)

2.1.2 Client (utilisateur connecté)

- Toutes les fonctionnalités du visiteur
- Passage de commandes en ligne
- Consultation de l'historique de ses commandes
- Annulation de commandes en attente
- Dépôt d'avis après livraison
- Export de ses données personnelles (RGPD)
- Suppression de son compte (droit à l'oubli)

2.1.3 Employé

- Toutes les fonctionnalités du client
- Consultation de toutes les commandes
- Modification du statut des commandes (validation, préparation, livraison, etc.)
- Validation et rejet des avis clients
- Création et modification de menus

2.1.4 Administrateur

- Toutes les fonctionnalités de l'employé
- Suppression de menus
- Gestion complète des plats (CRUD)
- Gestion des allergènes
- Gestion des utilisateurs (activation/désactivation, création de comptes employés)
- Consultation des statistiques (NoSQL)
- Suppression définitive d'avis

2.2 Cas d'usage principaux

Au total, j'ai identifié et implémenté **29 cas d'usage** couvrant l'ensemble des besoins fonctionnels.

2.2.1 Tableau complet des cas d'usage

ID	Acteur	Cas d'usage	Description	Priorité
UC01	Visiteur	Consulter les menus	Parcourir le catalogue avec filtres par thème et régime	HAUTE
UC02	Visiteur	Consulter le détail d'un menu	Voir la composition, allergènes, prix, avis	HAUTE
UC03	Visiteur	S'inscrire	Créer un compte avec validation stricte et consentement RGPD	HAUTE
UC04	Visiteur	Consulter politique RGPD	Lire la politique de confidentialité	MOYENNE

ID	Acteur	Cas d'usage	Description	Priorité
UC05	Visiteur	Consulter mentions légales	Accéder aux mentions légales	BASSE
UC06	Client	Se connecter	S'authentifier avec email et mot de passe	HAUTE
UC07	Client	Se déconnecter	Terminer sa session	HAUTE
UC08	Client	Passer une commande	Commander un menu avec date, adresse et quantité	HAUTE
UC09	Client	Consulter ses commandes	Voir l'historique avec statuts	HAUTE
UC10	Client	Consulter le détail d'une commande	Voir toutes les informations d'une commande	MOYENNE
UC11	Client	Annuler une commande	Annuler si statut « en attente »	MOYENNE
UC12	Client	Laisser un avis	Noter et commenter après livraison	MOYENNE
UC13	Client	Consulter ses avis	Voir les avis laissés	BASSE
UC14	Client	Exporter ses données	Télécharger au format JSON (RGPD)	HAUTE
UC15	Client	Supprimer son compte	Droit à l'oubli RGPD	HAUTE
UC16	Client	Gérer newsletter	Opt-in / opt-out	BASSE
UC17	Client	Contacter le service	Envoyer un message	BASSE
UC18	Employé	Consulter toutes les commandes	Vue d'ensemble avec filtres	HAUTE
UC19	Employé	Modifier statut commande	Workflow de validation	HAUTE
UC20	Employé	Consulter détail commande	Voir infos complètes	MOYENNE
UC21	Employé	Consulter avis en attente	Liste des avis à modérer	HAUTE
UC22	Employé	Valider un avis	Publier l'avis	HAUTE
UC23	Employé	Rejeter un avis	Masquer l'avis	HAUTE
UC24	Employé	Créer un menu	Nouveau menu avec plats	HAUTE
UC25	Employé	Modifier un menu	Éditer menu existant	HAUTE
UC26	Employé	Activer/désactiver menu	Bascule de visibilité	MOYENNE
UC27	Admin	Supprimer un menu	Suppression définitive	MOYENNE
UC28	Admin	Gérer plats	CRUD complet sur plats	HAUTE
UC29	Admin	Gérer allergènes	CRUD complet sur	MOYENNE

ID	Acteur	Cas d'usage	Description	Priorité
UC30	Admin	Gérer utilisateurs	allergènes Activer/désactiver comptes, créer employés	MOYENNE

2.2.2 UC08 : Passer une commande (priorité HAUTE)

Acteur principal : Client (utilisateur connecté)

Préconditions : l'utilisateur est connecté ; le menu sélectionné est actif ; le stock est suffisant.

Scénario nominal :

1. Le client consulte le détail d'un menu
2. Le client clique sur « Commander ce menu »
3. Le système affiche le formulaire de commande pré-rempli (adresse du profil, date minimum)
4. Le client saisit : date de livraison (future), adresse de livraison (modifiable), quantité ($\geq$ nb_personne_min), instructions (optionnel)
5. Le système calcule et affiche le **prix total en temps réel** (avec remise et frais de livraison éventuels)
6. Le client valide la commande
7. Le système vérifie la date (future), la quantité ($\geq$ minimum) et le stock (suffisant)
8. Le système crée la commande au statut « en attente »
9. Le système décrémente le stock du menu de manière atomique
10. Le système affiche une confirmation avec le numéro de commande
11. Le système **envoie un e-mail de confirmation de commande** (Resend)

Scénarios alternatifs :

- *3a. Date invalide (passée) :* message « La date doit être dans le futur », retour à l'étape 4.
- *3b. Quantité inférieure au minimum :* message « Minimum X personnes pour ce menu », retour à l'étape 4.
- *3c. Stock insuffisant :* message « Stock insuffisant (disponible : X) », proposition d'autres menus, fin du cas d'usage.

Postconditions : commande créée au statut « en attente », stock mis à jour, client redirigé vers « Mes commandes ».

Règles métier appliquées au calcul du prix :

- Prix de base : $\text{prix_base} \times \text{quantité}$
- **Remise automatique de 10 %** lorsque la quantité atteint au moins $\text{nb_personne_min} + 5$
- **Frais de livraison :** $5 \text{ €} + 0,59 \text{ €} \times \text{distance_km}$ (distance estimée depuis Bordeaux)
- **Prix total :** $(\text{prix_base} \times \text{quantité} \times (1 - \text{remise})) + \text{frais_livraison}$

Exigences non fonctionnelles : temps de réponse < 2 s ; calcul du prix en temps réel (JavaScript) ; validation côté client ET serveur.

2.2.3 UC11 : Annuler une commande (priorité MOYENNE)

Acteur principal : Client

Préconditions : l'utilisateur est connecté et possède une commande au statut « en attente ».

Scénario nominal : le client accède à « Mes commandes », clique sur « Annuler » pour une commande « en attente », confirme dans une modale ; le système passe le statut à « annulée », réincrémente le stock, horodate l'annulation, puis **envoie un e-mail de confirmation d'annulation**.

Scénarios alternatifs :

- *3a. Commande non annulable* : si la commande n'est plus « en attente », le bouton « Annuler » n'est pas affiché (« Cette commande ne peut plus être annulée »).
- *5a. Annulation échouée* : en cas d'erreur technique, message « Une erreur s'est produite », la commande reste inchangée, nouvelle tentative possible.

Postconditions : statut « annulée », stock restauré, commande conservée dans l'historique.

Règles métier : seules les commandes « en attente » sont annulables ; l'annulation restaure le stock de manière atomique ; la commande annulée reste visible (soft delete logique).

2.2.4 UC14 : Exporter ses données RGPD (priorité HAUTE)

Acteur principal : Client

Scénario nominal : depuis « Mes Données », le client clique sur « Télécharger mes données » ; le système génère un fichier JSON contenant ses informations de compte, ses commandes et ses avis, déclenche le téléchargement et trace l'action dans les logs.

Exemple de structure JSON exportée :

```
{
  "user": {
    "nom": "...", "prenom": "...", "email": "...",
    "gsm": "...", "adresse": "...", "role": "...",
    "created_at": "...", "consentement_rgpd": true, "newsletter":
false
  },
  "commandes": [
    { "id": 123, "menu": "Menu Bio", "date_commande": "...",
      "date_livraison": "...", "statut": "...", "prix_total": 175.00
    }
  ],
  "avis": [
    { "id": 45, "commande_id": 123, "note": 5, "commentaire": "...",
      "date": "..." }
  ]
}
```

Règles métier : toutes les données personnelles sont incluses ; le mot de passe n'est jamais exporté ; les horodatages sont au format ISO 8601 ; le format JSON assure la portabilité.

2.2.5 UC19 : Modifier le statut d'une commande (priorité HAUTE)

Acteur principal : Employé

Scénario nominal : l'employé ouvre le détail d'une commande depuis le dashboard, consulte les informations (client, menu, adresse, statut actuel), sélectionne le nouveau statut autorisé par le workflow, valide ; le système met à jour la commande, horodate le changement et **notifie le client par e-mail** selon le statut.

Workflow des statuts (8 statuts) :

```
en_attente
├──→ acceptée → en_préparation → en_livraison → livrée
└──
en_attente_retour_materiel → terminée
└──→ annulée (par le client tant que « en attente », ou par
l'employé)
```

i Les libellés exacts des statuts sont définis par des constantes dans le modèle Commande ; vérifier l'orthographe précise de chaque statut dans le code avant l'impression.

Règles métier : workflow globalement unidirectionnel ; seul le statut « en attente » est annulable par le client ; le passage en « en_attente_retour_materiel » déclenche un rappel concernant la **caution matériel de 600 €** (matériel à restituer sous 10 jours) ; « terminée » est le statut final et invite le client à laisser un avis ; chaque changement est horodaté.

3 Modélisation des données

L'application repose sur **deux bases de données complémentaires** :

- **PostgreSQL** (relationnel) : cœur transactionnel (utilisateurs, menus, commandes, avis...)
- **MongoDB Atlas** (NoSQL, orienté documents) : agrégats statistiques pour le tableau de bord (chiffre d'affaires et nombre de commandes par menu et par période)

Ce choix illustre la complémentarité d'un SGBD relationnel pour les données fortement structurées et d'une base documentaire pour des données analytiques dénormalisées et lues fréquemment.

3.1 Modèle Conceptuel de Données (MCD) – PostgreSQL

Le modèle relationnel repose sur des tables principales organisées de manière à respecter les formes normales et à assurer l'intégrité référentielle.

3.1.1 Entités principales

- **users** : utilisateurs (clients, employés, administrateurs)
- **menus** : menus proposés par le traiteur
- **plats** : plats composant les menus
- **allergenes** : liste des allergènes
- **commandes** : commandes passées par les clients
- **avis** : avis laissés par les clients

- **contacts** : messages de contact
- **menu_plat** : table associative (N:N entre menus et plats)
- **plat_allergene** : table associative (N:N entre plats et allergènes)

3.1.2 Relations principales

- Un utilisateur peut avoir plusieurs commandes (1:N)
- Un utilisateur peut laisser plusieurs avis (1:N)
- Un menu peut être commandé plusieurs fois (1:N)
- Une commande peut avoir un avis (1:1)
- Un menu contient plusieurs plats (N:N via menu_plat)
- Un plat peut contenir plusieurs allergènes (N:N via plat_allergene)

3.2 Structure de la table Users

users

—	id (BIGINT, PK)
—	nom (VARCHAR(100), NOT NULL)
—	prenom (VARCHAR(100), NOT NULL)
—	email (VARCHAR(255), UNIQUE, NOT NULL)
—	password (VARCHAR(255), NOT NULL)
—	gsm (VARCHAR(20), NOT NULL)
—	adresse (TEXT, NOT NULL)
—	role (ENUM: visiteur, utilisateur, employe, administrateur)
—	active (BOOLEAN, DEFAULT: true)
—	consentement_rgp (BOOLEAN, DEFAULT: false)
—	date_consentement (TIMESTAMP)
—	newsletter (BOOLEAN, DEFAULT: false)
—	created_at (TIMESTAMP)
—	updated_at (TIMESTAMP)

3.3 Normalisation

Le modèle respecte la 3^e forme normale (3NF) :

- **1NF** : toutes les valeurs sont atomiques (pas de tableaux ni de listes)
- **2NF** : toutes les colonnes dépendent de la clé primaire complète
- **3NF** : pas de dépendance transitive entre colonnes

Exemples : les allergènes sont dans une table dédiée (pas un champ texte), les plats sont réutilisables dans plusieurs menus, les relations N:N passent par des tables associatives.

3.4 Schéma de base de données complet (PostgreSQL)

3.4.1 Table users

```
CREATE TABLE users (
  id BIGSERIAL PRIMARY KEY,
  nom VARCHAR(100) NOT NULL,
  prenom VARCHAR(100) NOT NULL,
  email VARCHAR(255) UNIQUE NOT NULL,
  password VARCHAR(255) NOT NULL,
  gsm VARCHAR(20) NOT NULL,
  adresse TEXT NOT NULL,
  role VARCHAR(50) NOT NULL DEFAULT 'utilisateur'
```

```

    CHECK (role IN ('visiteur', 'utilisateur', 'employe',
'administrateur'))),
    active BOOLEAN DEFAULT TRUE,
    consentement_rgpd BOOLEAN DEFAULT FALSE,
    date_consentement TIMESTAMP,
    newsletter BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_users_role ON users(role);
CREATE INDEX idx_users_active ON users(active);

```

3.4.2 Table menus

```

CREATE TABLE menus (
    id BIGSERIAL PRIMARY KEY,
    titre VARCHAR(255) NOT NULL,
    description TEXT NOT NULL,
    theme VARCHAR(100) NOT NULL,
    regime VARCHAR(100) NOT NULL,
    nb_personne_min INTEGER NOT NULL CHECK (nb_personne_min > 0),
    prix_base DECIMAL(10,2) NOT NULL CHECK (prix_base > 0),
    stock INTEGER NOT NULL DEFAULT 0 CHECK (stock >= 0),
    conditions TEXT,
    actif BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX idx_menus_actif ON menus(actif);
CREATE INDEX idx_menus_theme ON menus(theme);
CREATE INDEX idx_menus_regime ON menus(regime);
CREATE INDEX idx_menus_prix ON menus(prix_base);

```

3.4.3 Table plats

```

CREATE TABLE plats (
    id BIGSERIAL PRIMARY KEY,
    nom VARCHAR(255) NOT NULL,
    description TEXT NOT NULL,
    categorie VARCHAR(100) NOT NULL
    CHECK (categorie IN ('entree', 'plat', 'dessert',
'accompagnement'))),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX idx_plats_categorie ON plats(categorie);

```

3.4.4 Table allergenes

```

CREATE TABLE allergenes (
    id BIGSERIAL PRIMARY KEY,
    nom VARCHAR(100) UNIQUE NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

3.4.5 Table commandes

```
CREATE TABLE commandes (  
  id BIGSERIAL PRIMARY KEY,  
  user_id BIGINT NOT NULL,  
  menu_id BIGINT NOT NULL,  
  date_commande TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  date_livraison DATE NOT NULL,  
  adresse_livraison TEXT NOT NULL,  
  quantite INTEGER NOT NULL CHECK (quantite > 0),  
  prix_total DECIMAL(10,2) NOT NULL CHECK (prix_total > 0),  
  prix_livraison DECIMAL(10,2) DEFAULT 0,  
  remise DECIMAL(5,2) DEFAULT 0,  
  statut VARCHAR(50) NOT NULL DEFAULT 'en_attente',  
  instructions TEXT,  
  motif_annulation TEXT,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,  
  FOREIGN KEY (menu_id) REFERENCES menus(id) ON DELETE RESTRICT  
);  
CREATE INDEX idx_commandes_user ON commandes(user_id);  
CREATE INDEX idx_commandes_menu ON commandes(menu_id);  
CREATE INDEX idx_commandes_statut ON commandes(statut);  
CREATE INDEX idx_commandes_date_livraison ON  
commandes(date_livraison);
```

3.4.6 Table avis

```
CREATE TABLE avis (  
  id BIGSERIAL PRIMARY KEY,  
  user_id BIGINT NOT NULL,  
  commande_id BIGINT NOT NULL UNIQUE,  
  note INTEGER NOT NULL CHECK (note >= 1 AND note <= 5),  
  commentaire TEXT NOT NULL,  
  statut VARCHAR(50) NOT NULL DEFAULT 'en_attente'  
  CHECK (statut IN ('en_attente', 'valide', 'rejete')),  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,  
  FOREIGN KEY (commande_id) REFERENCES commandes(id) ON DELETE  
CASCADE  
);  
CREATE INDEX idx_avis_user ON avis(user_id);  
CREATE INDEX idx_avis_commande ON avis(commande_id);  
CREATE INDEX idx_avis_statut ON avis(statut);  
CREATE INDEX idx_avis_note ON avis(note);
```

3.4.7 Tables associatives

-- Relation Menu <-> Plat (N:N)

```
CREATE TABLE menu_plat (  
  menu_id BIGINT NOT NULL,  
  plat_id BIGINT NOT NULL,  
  PRIMARY KEY (menu_id, plat_id),  
  FOREIGN KEY (menu_id) REFERENCES menus(id) ON DELETE CASCADE,  
  FOREIGN KEY (plat_id) REFERENCES plats(id) ON DELETE CASCADE
```

```
);
```

```
-- Relation Plat <-> Allergène (N:N)
```

```
CREATE TABLE plat_allergene (  
    plat_id BIGINT NOT NULL,  
    allergene_id BIGINT NOT NULL,  
    PRIMARY KEY (plat_id, allergene_id),  
    FOREIGN KEY (plat_id) REFERENCES plats(id) ON DELETE CASCADE,  
    FOREIGN KEY (allergene_id) REFERENCES allergenes(id) ON DELETE  
CASCADE  
);
```

3.5 Modèle NoSQL (MongoDB)

En complément du relationnel, une base **MongoDB Atlas** stocke des **statistiques agrégées** servant au tableau de bord d'administration. La collection `commandes_stats` (base `vite_gourmand_stats`) reçoit un document par commande, écrit automatiquement par un Observer Laravel à chaque création ou modification. Les statistiques (nombre de commandes et chiffre d'affaires par menu) sont calculées à la lecture par un pipeline d'agrégation.

Document type de la collection `commandes_stats` :

```
{  
  "commande_id": 22,  
  "menu_id": 2,  
  "menu_titre": "Menu Gastronomique",  
  "nb_personnes": 2,  
  "montant": 105.00,  
  "statut": "en_cours_livraison",  
  "date_commande": "2026-09-26T11:55:00Z"  
}  
// Pipeline d'agrégation (AdminStatsController)  
$match -> filtre facultatif par menu ou par période  
$group -> _id: "$menu_id", nb_commandes: { $sum: 1 },  
chiffre_affaires: { $sum: "$montant" }  
$sort -> chiffre_affaires décroissant
```

Côté Laravel, un modèle dédié (`CommandeStat`) utilise le connecteur MongoDB pour interroger cette collection. Ce choix dénormalisé évite des agrégations relationnelles coûteuses à chaque affichage du dashboard et illustre l'usage pertinent d'une base documentaire.

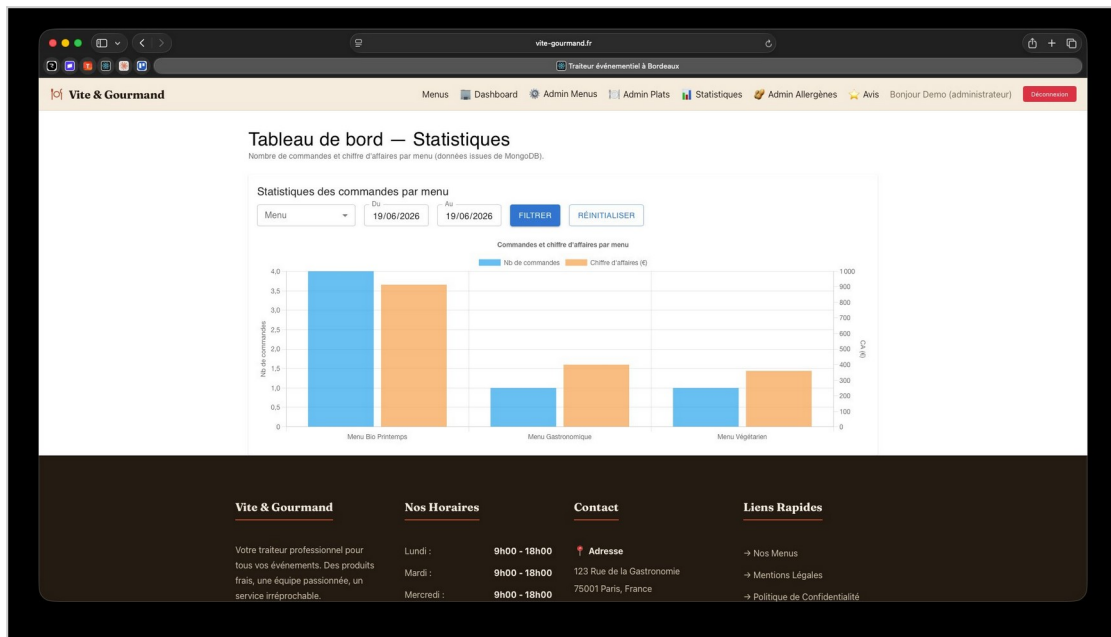
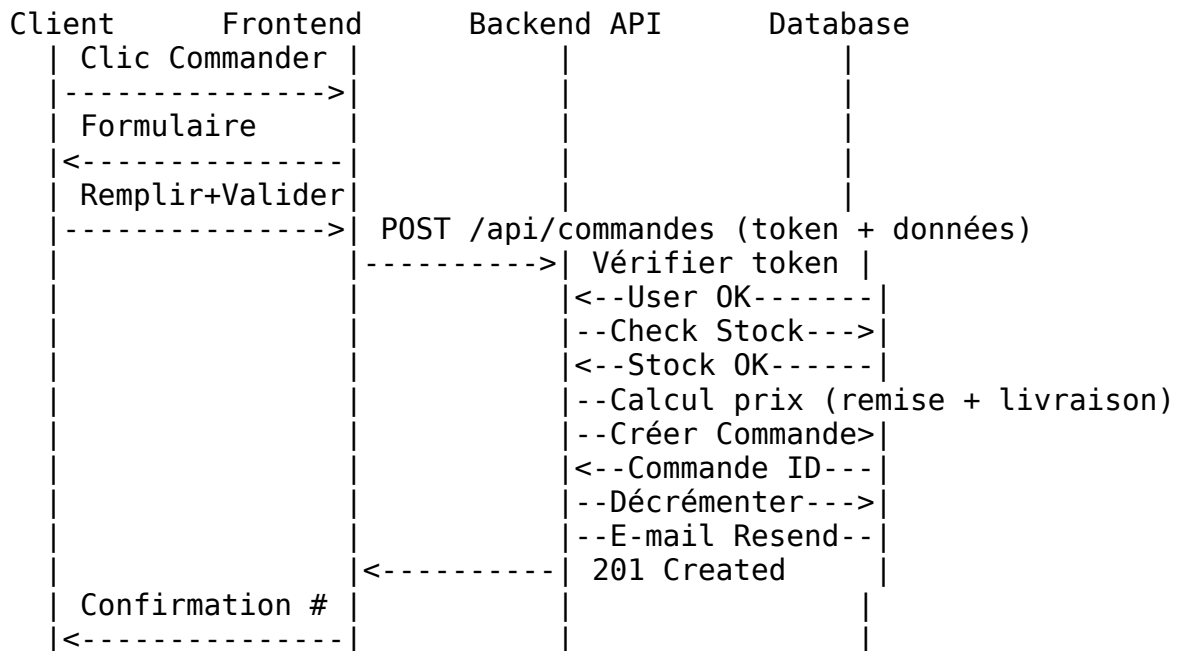


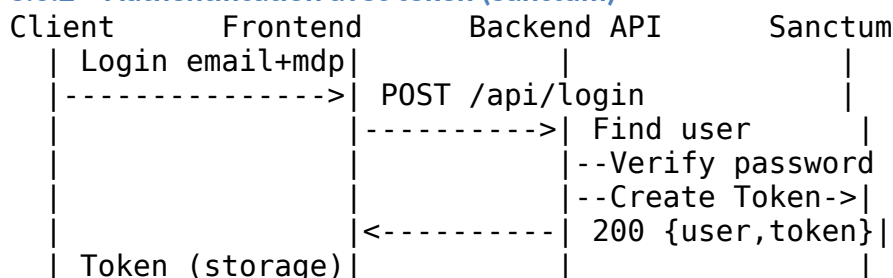
Figure : Tableau de bord des statistiques – graphique du chiffre d'affaires et du nombre de commandes par menu (données issues de MongoDB).

3.6 Diagrammes de séquence

3.6.1 Passer une commande



3.6.2 Authentification avec token (Sanctum)



```

|<-----|
| Requête (Bearer token) |
|----->| Verify token |
|<-----| 200 {data} |

```

3.6.3 Export des données RGPD

Client	Frontend	Backend API	Database
Clic Export			
	----->	GET /api/rgpd/export-data (token)	
		----->	Get User----->
			Get Commandes->
			Get Avis----->
			Build JSON
			Log Export----->
		<-----	200 {user, commandes, avis}
Téléchargement	fichier.json		
<-----			

4 Architecture technique

Conception : maquettes

Les maquettes ci-dessous ont été réalisées sous Figma avant le développement, afin de valider la structure des écrans et l'identité visuelle avec le client. Elles respectent la charte graphique présentée plus bas.

Écran d'accueil : présentation du traiteur, appels à l'action (découvrir les menus, s'inscrire) et pied de page.

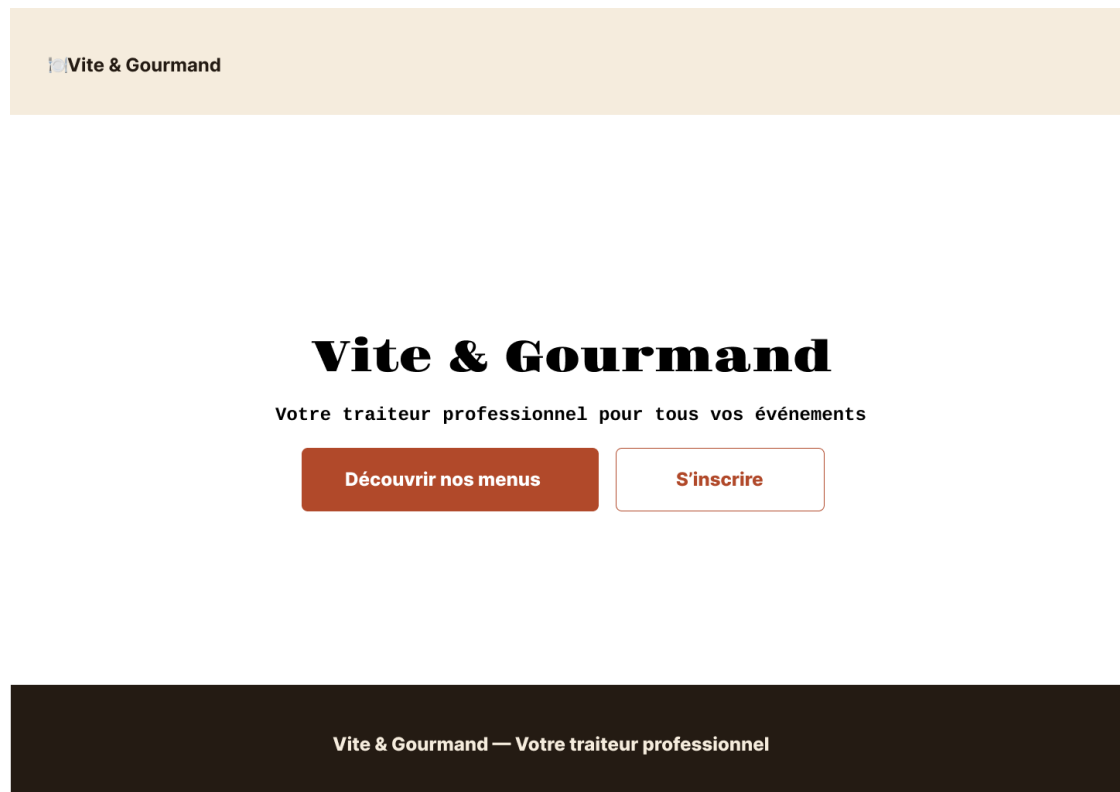


Figure : Maquette de la page d'accueil (Figma).

Page « Nos Menus » : catalogue des menus sous forme de cartes réutilisables (nom, description, prix, bouton de détail).

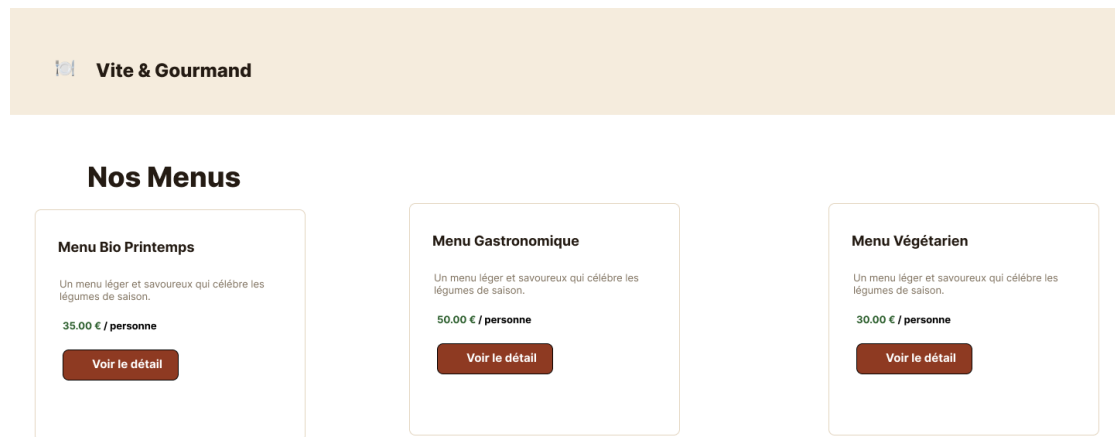


Figure : Maquette de la page des menus (Figma).

Charte graphique

L'identité visuelle de Vite & Gourmand repose sur des tons chaleureux et artisanaux, cohérents avec l'univers de la gastronomie.

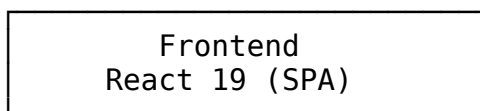
Couleurs principales : Crème #F5ECDD (fond), Terracotta #B1492A (boutons et accents), Brun foncé #241B13 (texte et pied de page), Beige #E3D5BF (bordures et séparateurs).

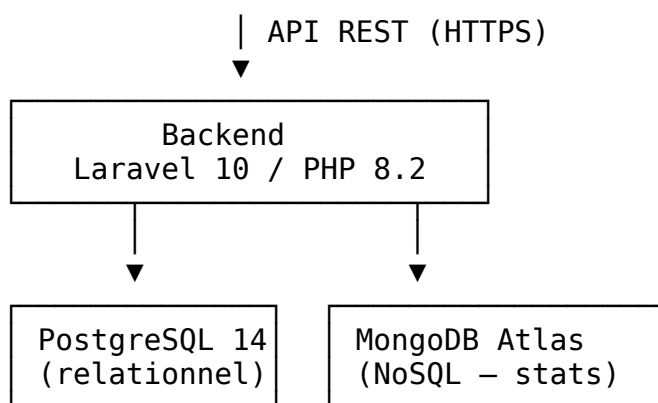
Typographie : la police Fraunces (à empattements) est utilisée pour les titres et le logo, apportant un caractère artisanal ; une police système sans empattement assure la lisibilité du corps de texte.

Logo : un pictogramme « couvert » associé au nom « Vite & Gourmand », décliné sur fond crème comme sur fond brun foncé.

4.1 Vue d'ensemble

J'ai opté pour une architecture **3-tiers découplée**, avec un frontend SPA, un backend API REST et une double persistance (relationnelle + NoSQL).





Serveur : Nginx sur VPS OVH (Ubuntu 22.04)
E-mails : Resend (API d'envoi transactionnel)

4.2 Choix techniques justifiés

4.2.1 Frontend : React 19

Justification : framework moderne et largement utilisé dans l'industrie ; composants réutilisables facilitant la maintenance ; écosystème riche (React Router, Axios, etc.) ; expérience utilisateur fluide (Single Page Application) ; performances optimales grâce au Virtual DOM.

Bibliothèques utilisées : React Router v6 (navigation et gestion des routes), Axios (communication HTTP avec le backend), Context API (gestion d'état global pour l'authentification).

4.2.2 Backend : Laravel 10

Justification : framework PHP robuste et mature ; Eloquent ORM pour une gestion élégante de la base ; Laravel Sanctum pour l'authentification API par tokens ; système de validation et de sécurité intégré ; documentation complète et communauté active.

Composants Laravel utilisés : Sanctum (authentification API), FormRequests (validation des données entrantes), middleware CheckRole et Policies (contrôle d'accès), Migrations (schéma de base), Notifications / Mail (e-mails Resend).

4.2.3 Bases de données : PostgreSQL 14 et MongoDB Atlas

PostgreSQL : SGBD relationnel performant et fiable ; respect strict des standards SQL ; types riches ; excellent support des contraintes d'intégrité ; performances optimales sur les requêtes complexes.

MongoDB Atlas : base documentaire infogérée pour les agrégats statistiques ; lecture rapide de données dénormalisées ; scalabilité horizontale.

4.2.4 Serveur web : Nginx

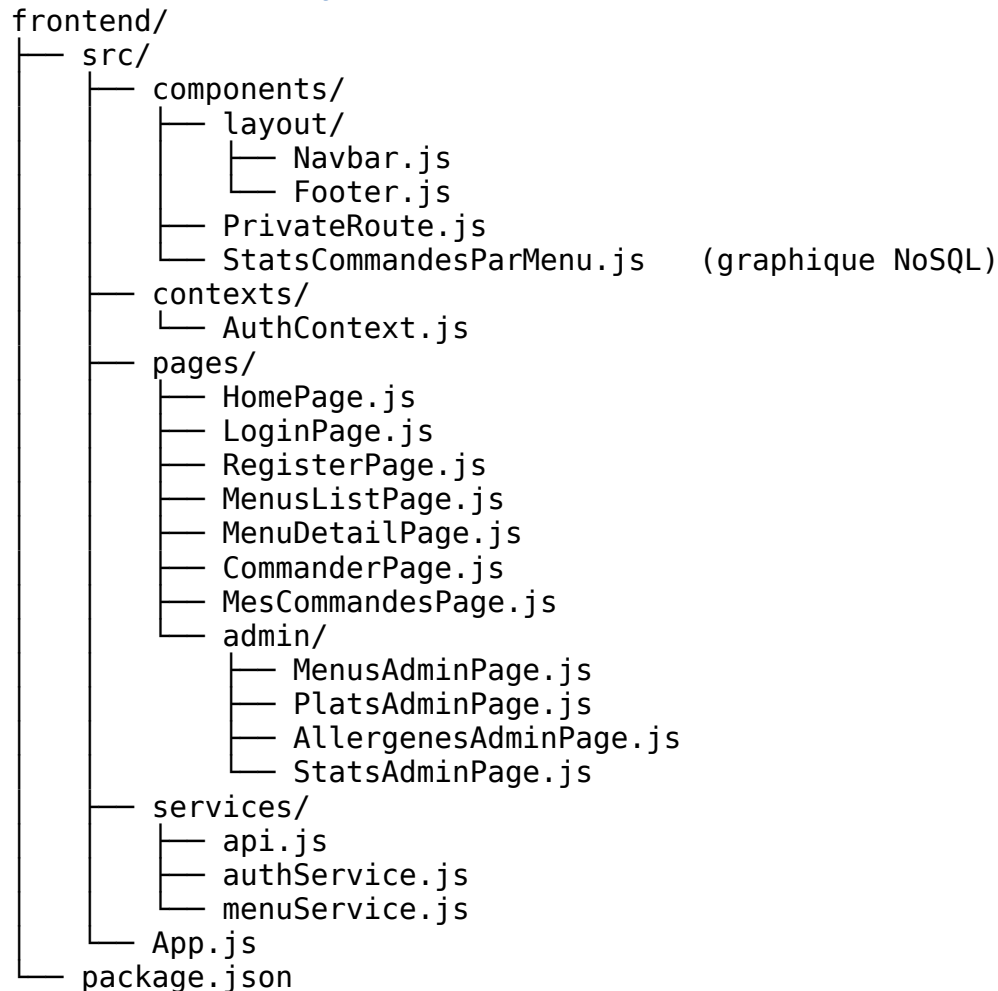
Serveur web performant et léger ; excellent proxy inverse ; configuration flexible ; gestion native de HTTPS.

4.2.5 Envoi d'e-mails : Resend

Service d'envoi transactionnel moderne (API + domaine vérifié), intégré à Laravel pour les notifications clients et employés.

4.3 Architecture frontend

4.3.1 Structure du projet React



4.3.2 Routing

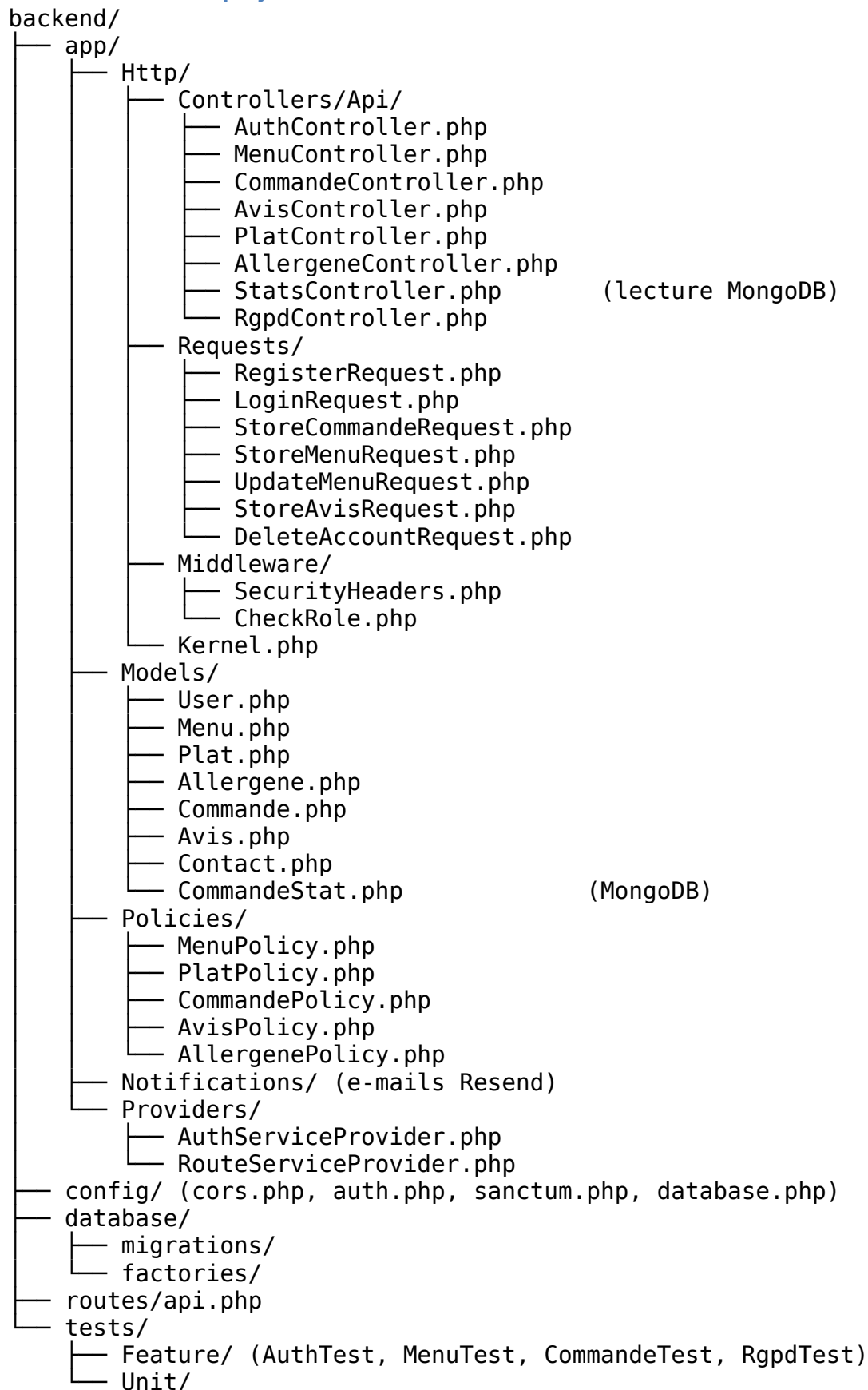
- **Routes publiques** : /, /login, /register, /menus, /menus/:id
- **Routes protégées (client)** : /commander/:id, /mes-commandes, /mes-donnees
- **Routes protégées (employé)** : /dashboard-employe, /admin/avis
- **Routes protégées (admin)** : /admin/menus, /admin/plats, /admin/allergenes, /admin/stats

4.3.3 Gestion de l'authentification

1. **Context API** : stockage de l'état utilisateur
2. **LocalStorage** : persistance du token
3. **Intercepteurs Axios** : ajout automatique du token aux requêtes
4. **PrivateRoute** : protection des routes selon le rôle

4.4 Architecture backend

4.4.1 Structure du projet Laravel



4.4.2 Routes API complètes

4.4.2.1 Routes publiques (5 routes)

```
GET /api/menus // Liste des menus actifs
GET /api/menus/{id} // Détail d'un menu
POST /api/register // Inscription
POST /api/login // Connexion
GET /api/rgpd/politique-confidentialite // Politique RGPD
```

4.4.2.2 Routes authentifiées – Client (15 routes)

```
POST /api/logout // Déconnexion
GET /api/me // Utilisateur connecté
GET /api/commandes // Mes commandes
POST /api/commandes // Créer une commande
GET /api/commandes/{id} // Détail commande
PUT /api/commandes/{id}/cancel // Annuler commande
POST /api/avis // Laisser un avis
GET /api/avis/commande/{commandeId} // Avis d'une commande
GET /api/rgpd/export-data // Export données JSON
DELETE /api/rgpd/delete-account // Supprimer compte
PUT /api/rgpd/newsletter // Gérer newsletter
POST /api/contact // Envoyer message
GET /api/mentions-legales // Mentions légales
```

4.4.2.3 Routes employé (12 routes)

```
GET /api/employe/dashboard // Statistiques employé
GET /api/admin/commandes // Toutes les commandes
GET /api/admin/commandes/{id} // Détail commande
PUT /api/admin/commandes/{id} // Modifier statut
GET /api/admin/avis/pending // Avis en attente
PUT /api/admin/avis/{id}/validate // Valider avis
PUT /api/admin/avis/{id}/reject // Rejeter avis
POST /api/admin/menus // Créer menu
PUT /api/admin/menus/{id} // Modifier menu
GET /api/admin/menus // Liste admin menus
PUT /api/admin/menus/{id}/toggle // Activer/désactiver
GET /api/admin/plats // Liste plats
```

4.4.2.4 Routes administrateur (14 routes)

```
DELETE /api/admin/menus/{id} // Supprimer menu
POST /api/admin/plats // Créer plat
GET /api/admin/plats/{id} // Détail plat
PUT /api/admin/plats/{id} // Modifier plat
DELETE /api/admin/plats/{id} // Supprimer plat
GET /api/admin/allergenes // Liste allergènes
POST /api/admin/allergenes // Créer allergène
PUT /api/admin/allergenes/{id} // Modifier allergène
DELETE /api/admin/allergenes/{id} // Supprimer allergène
GET /api/admin/users // Liste utilisateurs
PUT /api/admin/users/{id}/toggle // Activer/désactiver
DELETE /api/admin/avis/{id} // Supprimer avis
GET /api/admin/dashboard // Statistiques admin
GET /api/admin/stats // Statistiques NoSQL (MongoDB)
```

4.4.3 Exemples de contrôleurs

AuthController – Inscription :

```
public function register(RegisterRequest $request)
{
    $user = User::create([
        'nom' => $request->nom,
        'prenom' => $request->prenom,
        'email' => $request->email,
        'password' => Hash::make($request->password),
        'gsm' => $request->gsm,
        'adresse' => $request->adresse,
        'role' => 'utilisateur',
        'active' => true,
        'consentement_rgpd' => true,
        'date_consentement' => now(),
        'newsletter' => $request->newsletter ?? false,
    ]);

    $token = $user->createToken('auth-token')->plainTextToken;

    // E-mail de bienvenue (Resend)
    $user->notify(new BienvenueNotification());

    return response()->json(['user' => $user, 'token' => $token],
201);
}
```

CommandeController – Création (avec règles métier) :

```
public function store(StoreCommandeRequest $request)
{
    $this->authorize('create', Commande::class);

    $menu = Menu::findOrFail($request->menu_id);

    if ($menu->stock < $request->quantite) {
        return response()->json(['message' => 'Stock insuffisant'],
400);
    }

    // Prix de base
    $prix = $menu->prix_base * $request->quantite;

    // Remise automatique de 10 % si quantité >= nb_personne_min + 5
    $remise = 0;
    if ($request->quantite >= $menu->nb_personne_min + 5) {
        $remise = 0.10;
        $prix = $prix * (1 - $remise);
    }

    // Frais de livraison : 5 € + 0,59 €/km
    $fraisLivraison = 5 + (0.59 * $request->distance_km);
}
```

```

    $prixTotal = $prix + $fraisLivraison;

    $commande = Commande::create([
        'user_id' => $request->user()->id,
        'menu_id' => $menu->id,
        'date_commande' => now(),
        'date_livraison' => $request->date_livraison,
        'adresse_livraison' => $request->adresse_livraison,
        'quantite' => $request->quantite,
        'remise' => $remise * 100,
        'prix_livraison' => $fraisLivraison,
        'prix_total' => $prixTotal,
        'statut' => 'en_attente',
        'instructions' => $request->instructions,
    ]);

    $menu->decrement('stock', $request->quantite);

    // E-mail de confirmation (Resend)
    $request->user()->notify(new
CommandeConfirmeNotification($commande));

    return response()->json(['commande' => $commande->load('menu')],
201);
}

```

i Le calcul de la distance (distance_km) et le détail de la caution matériel sont à aligner précisément avec l'implémentation réelle ; les valeurs (5 €, 0,59 €/km, remise 10 %, caution 600 €) sont celles de tes règles métier.

MenuController – Liste avec filtres :

```

public function index(Request $request)
{
    $query = Menu::where('actif', true);

    if ($request->has('theme')) { $query->where('theme',
$request->theme); }
    if ($request->has('regime')) { $query->where('regime',
$request->regime); }
    if ($request->has('prix_min')) { $query->where('prix_base',
'>=', $request->prix_min); }
    if ($request->has('prix_max')) { $query->where('prix_base',
'<=', $request->prix_max); }

    $menus = $query->with(['plats.allergenes', 'avis' => function
($q) {
        $q->where('statut', 'valide');
    }])->paginate(12);

    return response()->json($menus);
}

```

4.4.4 Modèles Eloquent détaillés

Modèle User :

```
<?php
namespace App\Models;

use Illuminate\Foundation\Auth\User as Authenticatable;
use Laravel\Sanctum\HasApiTokens;

class User extends Authenticatable
{
    use HasApiTokens;

    protected $fillable = [
        'nom', 'prenom', 'email', 'password', 'gsm', 'adresse',
        'role', 'active', 'consentement_rgpd', 'date_consentement',
        'newsletter',
    ];

    protected $hidden = ['password', 'remember_token'];

    protected $casts = [
        'email_verified_at' => 'datetime',
        'date_consentement' => 'datetime',
        'active' => 'boolean',
        'consentement_rgpd' => 'boolean',
        'newsletter' => 'boolean',
    ];

    // Relations
    public function commandes() { return $this->hasMany(Commande::class); }
    public function avis() { return $this->hasMany(Avis::class); }

    // Accesseurs
    public function getNomCompleetAttribute()
    {
        return "{$this->prenom} {$this->nom}";
    }

    // Scopes
    public function scopeActif($query) { return $query->where('active', true); }
    public function scopeByRole($query, $role) { return $query->where('role', $role); }

    // Méthodes helper
    public function isAdmin() { return $this->role === 'administrateur'; }
    public function isEmploye() { return in_array($this->role, ['employe', 'administrateur']); }
    public function canManageCommandes() { return $this->
```

```
>isEmploye(); }
}
```

Modèle Menu :

```
<?php
namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Menu extends Model
{
    protected $fillable = [
        'titre', 'description', 'theme', 'regime',
        'nb_personne_min', 'prix_base', 'stock', 'conditions',
        'actif',
    ];

    protected $casts = [
        'prix_base' => 'decimal:2',
        'stock' => 'integer',
        'nb_personne_min' => 'integer',
        'actif' => 'boolean',
    ];

    protected $appends = ['note_moyenne'];

    // Relations
    public function plats() { return $this->belongsToMany(Plat::class, 'menu_plat'); }
    public function commandes() { return $this->hasMany(Commande::class); }
    public function avis() { return $this->hasManyThrough(Avis::class, Commande::class); }

    // Accesseurs
    public function getNoteMoyenneAttribute()
    {
        return $this->avis()->where('statut', 'valide')->avg('note')
        ?? 0;
    }

    public function getAllergenesAttribute()
    {
        return $this->plats()->with('allergenes')->get()
            ->pluck('allergenes')->flatten()->unique('id')-
            >values();
    }

    // Scopes
    public function scopeActif($query) { return $query->where('actif', true); }
    public function scopeByTheme($query, $theme) { return $query->where('theme', $theme); }
```

```

    public function scopeByRegime($query, $regime) { return $query-
>where('regime', $regime); }

```

```

    // Méthodes helper
    public function hasStock($quantite) { return $this->stock >=
$quantite; }
    public function decrementStock($quantite) { $this-
>decrement('stock', $quantite); }
}

```

Modèle Commande :

```

<?php
namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Commande extends Model
{
    protected $fillable = [
        'user_id', 'menu_id', 'date_commande', 'date_livraison',
        'adresse_livraison', 'quantite', 'prix_total',
        'prix_livraison',
        'remise', 'statut', 'instructions', 'motif_annulation',
    ];

    protected $casts = [
        'date_commande' => 'datetime',
        'date_livraison' => 'date',
        'prix_total' => 'decimal:2',
        'quantite' => 'integer',
    ];

    const STATUT_EN_ATTENTE = 'en_attente';
    const STATUT_ACCEPTEE = 'acceptee';
    const STATUT_EN_PREPARATION = 'en_preparation';
    const STATUT_EN_LIVRAISON = 'en_livraison';
    const STATUT_LIVREE = 'livree';
    const STATUT_RETOUR_MATERIEL = 'en_attente_retour_materiel';
    const STATUT_TERMINEE = 'terminee';
    const STATUT_ANNULEE = 'annulee';

    // Relations
    public function user() { return $this->belongsTo(User::class); }
    public function menu() { return $this->belongsTo(Menu::class); }
    public function avis() { return $this->hasOne(Avis::class); }

    // Scopes
    public function scopeEnAttente($query) { return $query-
>where('statut', self::STATUT_EN_ATTENTE); }
    public function scopeLivrees($query) { return $query-
>where('statut', self::STATUT_LIVREE); }

    // Méthodes helper

```

```

        public function canBeCancelled() { return $this->statut ===
self::STATUT_EN_ATTENTE; }
        public function canHaveAvis() { return $this->statut ===
self::STATUT_TERMINEE && !$this->avis; }
        public function updateStatut($nouveau) { $this->update(['statut'
=> $nouveau]); }
    }

```

4.4.5 FormRequests - validation détaillée

RegisterRequest :

```

<?php
namespace App\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Validation\Rules\Password;

class RegisterRequest extends FormRequest
{
    public function authorize() { return true; }

    public function rules()
    {
        return [
            'nom' => ['required', 'string', 'max:100',
'regex:/^[a-zA-ZÀ-ÿ\s\-\-]+$/'],
            'prenom' => ['required', 'string', 'max:100',
'regex:/^[a-zA-ZÀ-ÿ\s\-\-]+$/'],
            'email' => ['required', 'string', 'email:rfc,dns',
'max:255', 'unique:users,email'],
            'password' => ['required', 'string', 'confirmed',
Password::min(10)->mixedCase()->numbers()-
>symbols()->uncompromised()],
            'gsm' => ['required', 'string', 'regex:/^\+?[0-9]
{10,15}$/'],
            'adresse' => ['required', 'string', 'max:500'],
            'consentement_rgpd' => ['required', 'accepted'],
            'newsletter' => ['boolean'],
        ];
    }

    public function messages()
    {
        return [
            'nom.required' => 'Le nom est obligatoire.',
            'nom.regex' => 'Le nom ne peut contenir que des
lettres.',
            'email.unique' => 'Cet email est déjà utilisé.',
            'password.min' => 'Le mot de passe doit contenir au
moins 10 caractères.',
            'consentement_rgpd.accepted' => 'Vous devez accepter la
politique RGPD.',
        ];
    }
}

```

```

    }
}

StoreCommandeRequest :

<?php
namespace App\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;

class StoreCommandeRequest extends FormRequest
{
    public function authorize() { return auth()->check(); }

    public function rules()
    {
        return [
            'menu_id' => ['required', 'integer', 'exists:menus,id'],
            'date_livraison' => ['required', 'date', 'after:today'],
            'adresse_livraison' => ['required', 'string',
'max:500'],
            'quantite' => ['required', 'integer', 'min:1',
'max:100'],
            'instructions' => ['nullable', 'string', 'max:1000'],
        ];
    }

    public function messages()
    {
        return [
            'menu_id.required' => 'Vous devez sélectionner un
menu.',
            'menu_id.exists' => 'Ce menu n\'existe pas.',
            'date_livraison.after' => 'La date de livraison doit
être dans le futur.',
            'quantite.min' => 'La quantité minimum est de 1.',
        ];
    }

    public function withValidator($validator)
    {
        $validator->after(function ($validator) {
            $menu = \App\Models\Menu::find($this->menu_id);
            if ($menu && $this->quantite < $menu->nb_personne_min) {
                $validator->errors()->add('quantite',
                    "La quantité minimum pour ce menu est de {$menu-
>nb_personne_min} personnes.");
            }
            if ($menu && $menu->stock < $this->quantite) {
                $validator->errors()->add('quantite',
                    "Stock insuffisant. Disponible : {$menu-
>stock}");
            }
        });
    }
}

```

```
}  
}
```

5 Sécurité

5.1 Authentification (Laravel Sanctum)

L'authentification par tokens est implémentée avec Laravel Sanctum :

```
// Génération du token au login  
$token = $user->createToken('auth-token')->plainTextToken;  
  
// Protection des routes  
Route::middleware('auth:sanctum')->group(function () {  
    // routes protégées  
});
```

5.1.1 Validation des mots de passe

Règles strictes : minimum 10 caractères, au moins une majuscule, une minuscule, un chiffre et un symbole, et vérification contre les bases de mots de passe compromis (uncompromised). Les mots de passe sont hachés avec bcrypt avant stockage.

5.2 Validation des données (FormRequests)

Toutes les données entrantes sont validées via des FormRequests (voir chapitre Architecture pour les implémentations complètes). Les règles couvrent le format, la longueur, l'unicité, la cohérence métier et renvoient des messages d'erreur explicites en français.

5.3 Contrôle d'accès (middleware de rôle + Policies)

Le contrôle d'accès repose sur un **middleware CheckRole** appliqué au groupe de routes d'administration, complété par des Policies pour certaines actions ciblées.

```
// Middleware variadique de contrôle de rôle  
public function handle(Request $request, Closure $next, ...$roles)  
{  
    if (! $request->user() || ! in_array($request->user()->role,  
$roles)) {  
        abort(403, 'Accès non autorisé.');    }  
    return $next($request);  
}  
  
// routes/api.php – groupe d'administration protégé  
Route::middleware([  
    'auth:sanctum',  
    'throttle:admin',  
    \App\Http\Middleware\CheckRole::class .  
    ':administrateur,employe',  
)->prefix('admin')->group(function () {  
    // création / modification / suppression de menus, plats,  
    // allergènes...  
});
```

```
// Exemple de Policy ciblée (MenuPolicy)
public function create(User $user)
{
    return in_array($user->role, ['administrateur', 'employe']);
}

public function delete(User $user, Menu $menu)
{
    return $user->role === 'administrateur';
}
```

Cette protection a été **validée par les tests automatisés** (voir chapitre Tests) : un test vérifie qu'un utilisateur simple obtient bien une réponse **403** lorsqu'il tente de créer un menu.

5.4 Gestion des secrets

- Les fichiers `.env` (et `.env.testing`) sont **exclus du dépôt Git** via `.gitignore` ; aucune information sensible n'est versionnée.
- Les identifiants de **MongoDB Atlas** sont stockés uniquement dans le `.env` du serveur ; le mot de passe a été **renouvelé** après vérification, et l'accès réseau Atlas est restreint par **liste d'autorisation d'IP**.
- Les clés d'API (Resend) sont également conservées hors du dépôt.

5.5 Protection du transport

- Certificat SSL Let's Encrypt, renouvellement automatique (Certbot)
- Redirection automatique HTTP → HTTPS, HSTS activé (Strict-Transport-Security)

5.5.1 Headers de sécurité

```
$response->headers->set('X-Frame-Options', 'DENY');
$response->headers->set('X-Content-Type-Options', 'nosniff');
$response->headers->set('X-XSS-Protection', '1; mode=block');
$response->headers->set('Content-Security-Policy', "...");
```

5.6 Rate limiting

- 5 tentatives de connexion par minute
- 60 requêtes API par minute pour les utilisateurs authentifiés
- 60 requêtes API par minute par IP pour les visiteurs
- Limiteur dédié `throttle:admin` sur les routes d'administration

5.7 CORS

```
'allowed_origins' => [
    'https://vite-gourmand.fr',
    'https://www.vite-gourmand.fr',
],
'supports_credentials' => true,
```

6 Conformité RGPD

6.1 Droits implémentés

6.1.1 Droit à l'information

- Politique de confidentialité accessible publiquement
- Mentions légales disponibles
- Consentement explicite obligatoire à l'inscription
- Information claire sur les données collectées

6.1.2 Droit d'accès et de portabilité

```
public function exportData(Request $request)
{
    $user = $request->user();
    return response()->json([
        'user' => $user,
        'commandes' => $user->commandes()->with('menu')->get(),
        'avis' => $user->avis()->with('commande')->get(),
    ]);
}
```

L'utilisateur peut télécharger toutes ses données au format JSON.

6.1.3 Droit à l'oubli

- Confirmation en 2 étapes (texte « SUPPRIMER » + mot de passe)
- Suppression définitive et irréversible
- Suppression en cascade : commandes, avis, tokens

6.2 Gestion du consentement

À l'inscription : case RGPD obligatoire, date de consentement enregistrée, opt-in newsletter explicite et optionnel. L'utilisateur peut à tout moment consulter ses consentements, modifier son opt-in, exporter ses données ou supprimer son compte.

6.3 Sécurité des données

Stockage sécurisé en base, mots de passe hachés (jamais en clair), connexion HTTPS uniquement, accès limité selon les rôles.

7 Notifications par e-mail (Resend)

Les e-mails transactionnels sont envoyés via **Resend** (domaine vérifié) et déclenchés par les événements métier :

- **E-mail de bienvenue** — à la création d'un compte utilisateur
- **Confirmation de commande** — récapitulatif (menu, prix, date de prestation)
- **Commande terminée** — invitation à laisser un avis
- **Rappel de retour matériel** — passage au statut « retour matériel » (caution de 600 €, restitution sous 10 jours)

- **Création de compte employé** — notification à l'employé (sans le mot de passe en clair)

L'intégration utilise le système de Notifications/Mailables de Laravel, ce qui permet de centraliser le contenu des messages et de les tester facilement.

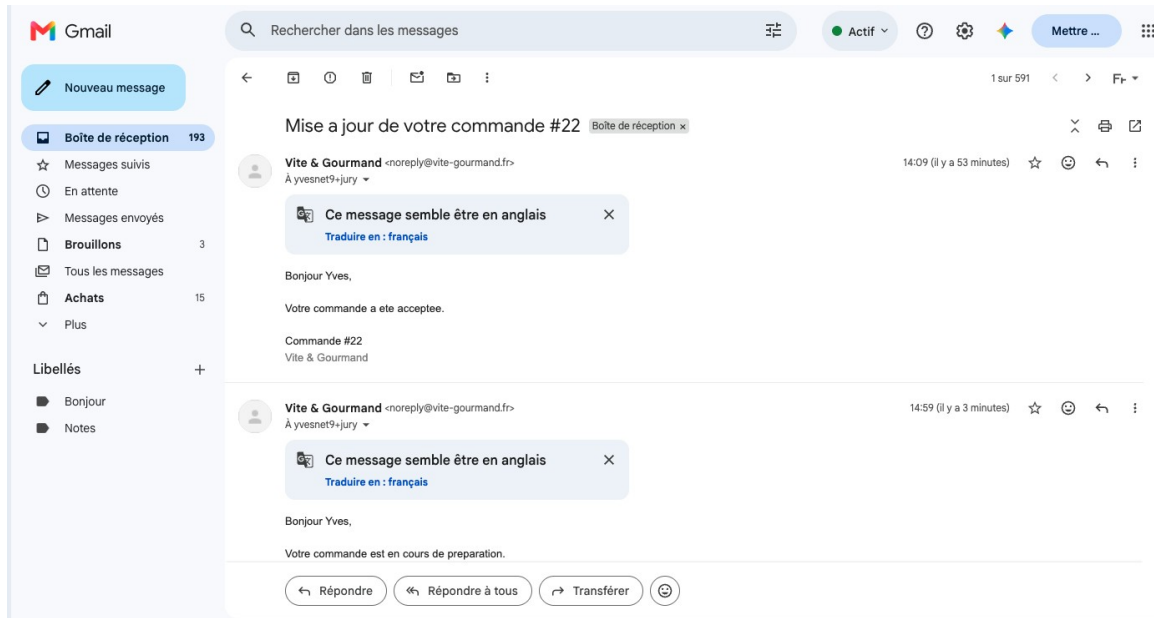


Figure 7.1 – E-mails transactionnels reçus (expéditeur `noreply@vite-gourmand.fr`) : confirmation de commande et mise à jour de statut.

8 Tests

8.1 Tests automatisés (PHPUnit)

J'ai développé **18 tests automatisés** couvrant les fonctionnalités critiques. Ils s'exécutent sur une base **SQLite en mémoire** (configuration `.env.testing`), garantissant un environnement isolé et reproductible, sans impact sur les données de production.

8.1.1 Tests d'authentification (RegisterTest, LoginTest)

```
test_user_can_register_with_valid_data()
test_user_cannot_register_without_rgpd_consent()
test_user_cannot_register_with_existing_email()
test_user_cannot_register_with_weak_password()
test_user_cannot_register_with_mismatched_passwords()
test_user_cannot_register_with_invalid_phone()
```

```
test_user_can_login_with_valid_credentials()
test_user_cannot_login_with_wrong_password()
test_user_can_logout()
```

8.1.2 Tests CRUD Menus / autorisations

```
test_guest_can_view_menus()
test_admin_can_create_menu()
test_regular_user_cannot_create_menu() // Vérifie le 403 (contrôle d'accès)
```

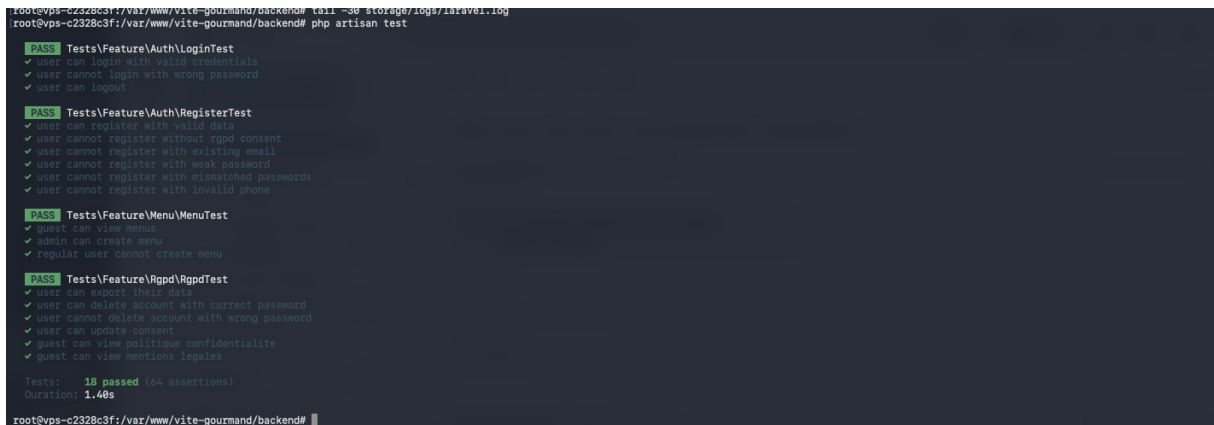
8.1.3 Tests RGPD (RgpdTest)

```
test_user_can_export_their_data()
test_user_can_delete_account_with_correct_password()
test_user_cannot_delete_account_with_wrong_password()
test_user_can_update_consent()
test_guest_can_view_politique_confidentialite()
test_guest_can_view_mentions_legales()
```

8.2 Exécution des tests

php artisan test

Résultat : 18 tests passés (64 assertions)



```
root@vps-c2328c3f:/var/www/vite-gourmand/backend# tail -30 storage/logs/laravel.log
root@vps-c2328c3f:/var/www/vite-gourmand/backend# php artisan test

PASS Tests\Feature\Auth\LoginTest
✓ user can login with valid credentials
✓ user cannot login with wrong password
✓ user can logout

PASS Tests\Feature\Auth\RegisterTest
✓ user can register with valid data
✓ user cannot register without rgpd consent
✓ user cannot register with existing email
✓ user cannot register with weak password
✓ user cannot register with mismatched passwords
✓ user cannot register with invalid phone

PASS Tests\Feature\Menu\MenuTest
✓ guest can view menu
✓ admin can create menu
✓ regular user cannot create menu

PASS Tests\Feature\Rgpd\RgpdTest
✓ user can export their data
✓ user can delete account with correct password
✓ user cannot delete account with wrong password
✓ user can update consent
✓ guest can view politique confidentialite
✓ guest can view mentions legales

Tests: 18 passed (64 assertions)
Duration: 1.40s

root@vps-c2328c3f:/var/www/vite-gourmand/backend#
```

Figure 8.1 – Exécution de « php artisan test » : 18 tests passés (64 assertions), dont « regular user cannot create menu » (contrôle de rôle).

8.3 Couverture fonctionnelle

Les tests couvrent l'authentification complète (inscription, connexion, déconnexion), la validation des données entrantes, les autorisations (middleware de rôle et Policies), les fonctionnalités RGPD et l'intégrité des données.

8.4 Apport des tests : détection d'une faille d'autorisation

L'exécution des tests a révélé une **faille de contrôle d'accès (broken access control)** : le groupe de routes d'administration imposait l'authentification mais **pas la vérification du rôle**. Le test « un utilisateur simple ne peut pas créer de menu » attendait un code **403** mais recevait **201** (création réussie).

Correction : création d'un middleware `CheckRole` appliqué au groupe d'administration, restreignant l'accès aux rôles `administrateur` et `employe`. Après correction, l'ensemble des **18 tests** passe. Cet incident illustre concrètement la valeur des tests automatisés pour la sécurité (principe de moindre privilège, OWASP A01 – Broken Access Control).

9 Déploiement en production

9.1 Infrastructure

9.1.1 Serveur VPS OVH

- Système : Ubuntu 22.04 LTS

- IP : 37.59.124.193
- Domaine : vite-gourmand.fr (+ www)
- Accès : SSH avec authentification par clé

9.1.2 Stack serveur

- Nginx 1.18.0 : serveur web et proxy inverse
- PHP 8.2 + PHP-FPM : traitement des requêtes PHP
- PostgreSQL 14 : base de données relationnelle
- MongoDB Atlas : base NoSQL (hébergée)
- Composer 2.x : gestionnaire de dépendances PHP
- Node.js 18 : build du frontend

9.2 Configuration Nginx

```
server {
    listen 443 ssl http2;
    server_name vite-gourmand.fr www.vite-gourmand.fr;

    # SSL
    ssl_certificate
/etc/letsencrypt/live/vite-gourmand.fr/fullchain.pem;
    ssl_certificate_key
/etc/letsencrypt/live/vite-gourmand.fr/privkey.pem;

    # Frontend React
    root /var/www/vite-gourmand/frontend/build;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }

    # Backend API Laravel
    location /api/ {
        root /var/www/vite-gourmand/backend/public;
        rewrite ^/api/(.*)$ /index.php?/$1 last;
    }

    # PHP processing
    location ~ \.php$ {
        root /var/www/vite-gourmand/backend/public;
        fastcgi_pass unix:/var/run/php/php8.2-fpm.sock;
        fastcgi_index index.php;
        include fastcgi_params;
        fastcgi_param SCRIPT_FILENAME
$document_root$fastcgi_script_name;
    }
}
```

9.3 Processus de déploiement

9.3.1 1. Préparation du serveur

```
sudo apt update
sudo apt install nginx php8.2-fpm php8.2-pgsql postgresql nodejs npm
```

```
sudo -u postgres psql
CREATE DATABASE vite_gourmand_prod;
CREATE USER vite_gourmand WITH PASSWORD 'mot_de_passe_securise';
GRANT ALL PRIVILEGES ON DATABASE vite_gourmand_prod TO
vite_gourmand;
```

9.3.2 2. Déploiement du backend

```
cd /var/www/vite-gourmand/backend
composer install --no-dev --optimize-autoloader
php artisan migrate --force
php artisan config:cache
php artisan route:cache
```

9.3.3 3. Build du frontend

```
cd /var/www/vite-gourmand/frontend
npm install
echo "REACT_APP_API_URL=https://vite-gourmand.fr/api"
> .env.production
npm run build
```

9.3.4 4. Installation SSL

```
sudo apt install certbot python3-certbot-nginx
sudo certbot --nginx -d vite-gourmand.fr -d www.vite-gourmand.fr
```

Le certificat se renouvelle automatiquement tous les 90 jours.

9.4 Mise en production

URL publique : <https://vite-gourmand.fr> — HTTPS actif, certificat valide.

Tests de production réalisés : inscription et connexion fonctionnelles, passage de commande opérationnel, dashboard employé accessible, dashboard administrateur fonctionnel, export RGPD opérationnel, statistiques NoSQL affichées, performance satisfaisante.

Route	Temps moyen	Statut
GET /api/menus	120 ms	Optimal
POST /api/commandes	180 ms	Bon
GET /api/admin/commandes	250 ms	À optimiser
POST /api/register	300 ms	Bcrypt (normal)

9.5 Configuration PHP-FPM

PHP-FPM est configuré en mode dynamique : le nombre de processus PHP s'ajuste automatiquement selon la charge, ce qui optimise l'utilisation de la mémoire du serveur. Pour la sécurité, la directive `expose_php` est désactivée afin de ne pas révéler la version de

PHP, et les limites de mémoire et de temps d'exécution sont adaptées aux besoins de l'application. OPcache est activé pour mettre en cache le code PHP compilé et accélérer l'exécution.

9.6 Configuration PostgreSQL

PostgreSQL est configuré pour l'usage de l'application : le nombre maximal de connexions et les paramètres mémoire sont ajustés pour de bonnes performances sans surcharger le serveur. L'accès à la base est restreint aux connexions locales (le serveur applicatif et la base sont sur la même machine), ce qui limite la surface d'attaque. Les identifiants de connexion sont stockés dans le fichier .env, hors du dépôt Git.

9.7 Optimisations de performance

Plusieurs optimisations ont été mises en place à chaque niveau de l'application :

- Backend Laravel : mise en cache de la configuration et des routes, autoloader optimisé (composer --optimize-autoloader), et eager loading des relations Eloquent pour éviter le problème des requêtes N+1.
- Frontend React : build de production minifié (minification du JavaScript, tree shaking, découpage du code), et mise en cache des fichiers statiques par Nginx. Le chargement différé des pages (React.lazy) est une piste d'amélioration.
- Base de données : index automatiques de PostgreSQL sur les clés primaires et les colonnes uniques (dont users.email, utilisé à chaque connexion), et chargement anticipé des relations (eager loading) pour limiter le nombre de requêtes. Pistes d'amélioration : index sur les clés étrangères les plus sollicitées et pagination des listes quand le volume augmentera.
- Nginx : compression gzip des réponses et mise en cache des fichiers statiques, pour réduire le volume transféré et le temps de chargement.

9.8 Monitoring et sauvegardes

Le suivi de l'application s'appuie sur les journaux (logs) de chaque service : Laravel (erreurs applicatives), Nginx (accès et erreurs web) et PostgreSQL (requêtes lentes). En production, l'utilisation des ressources reste modérée (CPU faible, mémoire maîtrisée).

Une sauvegarde automatique de la base de données est réalisée chaque nuit via une tâche planifiée (cron), et les sauvegardes sont conservées sur une période glissante. Cela garantit la possibilité de restaurer les données en cas d'incident.

10 Réalisation

10.1 Fonctionnalités développées

10.1.1 Espace public

Consultation des menus : liste paginée des menus actifs, filtres par thème et par régime alimentaire, affichage détaillé (composition, allergènes, prix, stock), système de notation visible (moyenne des avis validés).

Inscription et connexion : formulaire d'inscription avec validation stricte, vérification de la disponibilité de l'email, validation du mot de passe (force, correspondance), consentement RGPD obligatoire, opt-in newsletter optionnel, connexion sécurisée avec génération de token, rate limiting (5 tentatives/minute).

10.1.2 Espace client

Gestion des commandes : formulaire de commande (sélection du menu, date de livraison future, adresse pré-remplie modifiable, quantité avec minimum, instructions), calcul automatique du prix total (remise et frais de livraison), historique avec statuts colorés, annulation des commandes en attente, détail de chaque commande.

Système d'avis : note de 1 à 5 étoiles, commentaire, un seul avis par commande, modération avant publication, affichage des avis validés sur les menus.

RGPD : page « Mes Données » (export JSON, gestion du consentement newsletter, suppression de compte sécurisée en deux étapes).

10.1.3 Espace employé

Dashboard : statistiques (commandes en attente, avis à valider, chiffre d'affaires du jour), liste de toutes les commandes avec filtres, gestion du workflow des statuts, modération des avis (validation / rejet).

Gestion des menus : création (formulaire complet, sélection des plats, thème et régime, prix et stock), modification, activation/désactivation.

10.1.4 Espace administrateur

Dashboard admin : toutes les fonctionnalités employé, statistiques avancées (NoSQL), vue d'ensemble de la plateforme.

Gestion complète : menus (création, modification, suppression avec vérification des commandes associées), plats (CRUD, association aux allergènes), allergènes (création et gestion), utilisateurs (liste, activation/désactivation, création de comptes employés).

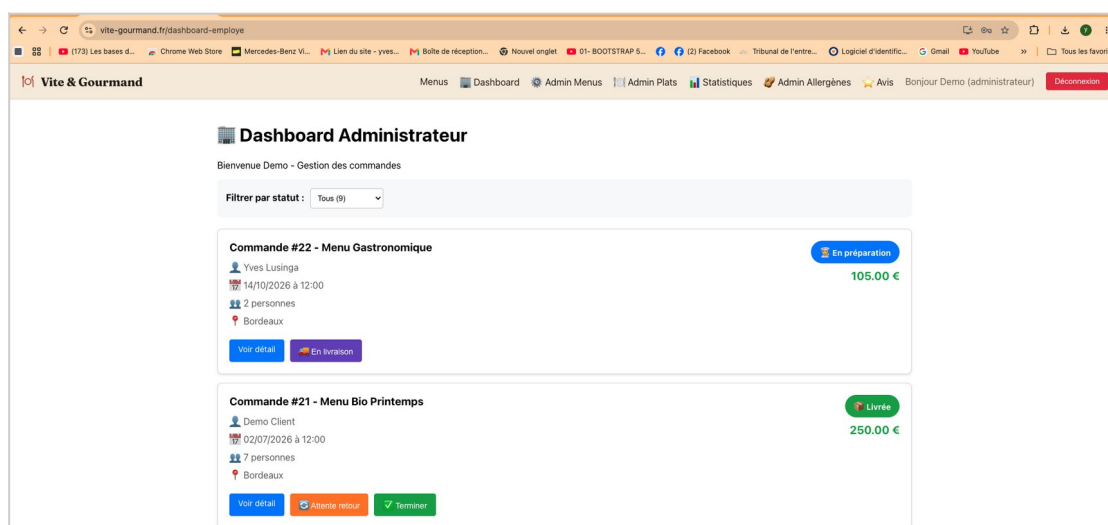


Figure : Tableau de bord administrateur – gestion des commandes et accès aux espaces d'administration.

10.2 Accessibilité (RGAA)

Une passe d'accessibilité a été réalisée sur les pages principales du front React, en visant les critères RGAA à plus fort impact :

- **Langue et titre de page** : `lang="fr"` sur le document, `<title>` pertinent (critères 8.3 / 8.5).
- **Formulaires** : chaque champ relié à son `<label>` via `for/id`, champs obligatoires indiqués, aide du mot de passe reliée par `aria-describedby`, attributs `autocomplete` (critère 11.1).
- **Messages d'erreur** : annoncés aux technologies d'assistance via `role="alert"` (critère 11.10).
- **Structure** : un seul `<h1>` par page et hiérarchie de titres respectée (critère 9.1), repères sémantiques `<nav>`, `<main>`, `<footer>`.
- **Navigation au clavier** : **lien d'évitement** « Aller au contenu principal » (critère 12.7), focus visibles.
- **Boutons et liens** : bouton de menu mobile doté d'un `aria-label` et d'un état `aria-expanded`, intitulés de liens explicites (critères 6.1 / 7.x).
- **Contrastes** : couleurs ajustées pour viser un ratio de 4,5:1 (critère 3.2).

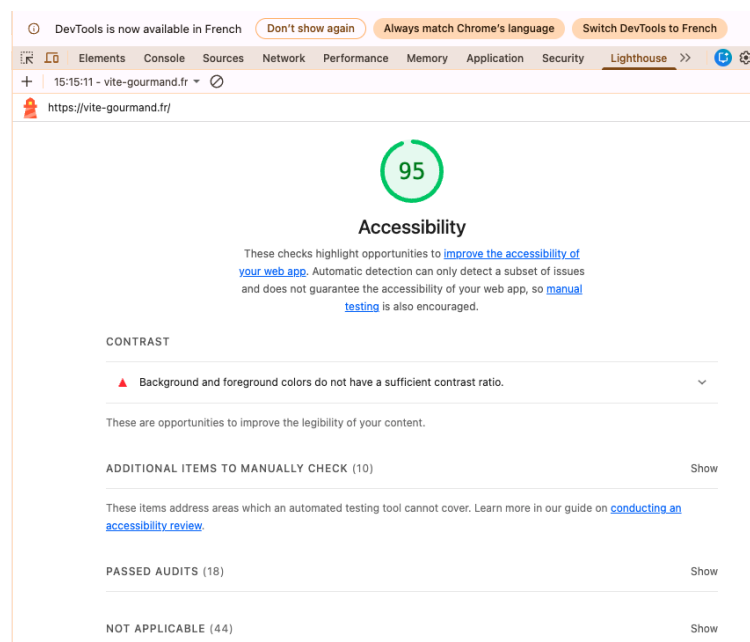


Figure 10.1 – Audit Lighthouse (accessibilité) de la page d'accueil vite-gourmand.fr : score de 95.

10.3 Captures d'écran des interfaces

Des captures d'écran de l'application en production sont insérées en annexe.

Les captures ci-dessous présentent les principales interfaces de l'application en production, du parcours visiteur jusqu'aux espaces d'administration.

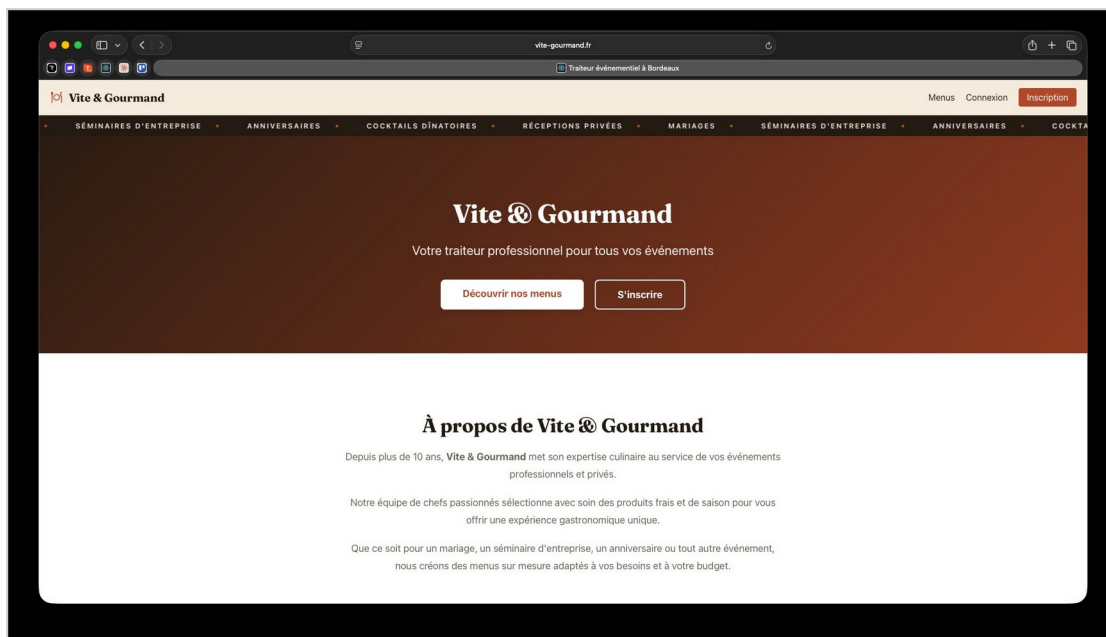


Figure A.1 – Page d'accueil du site en production (vite-gourmand.fr).

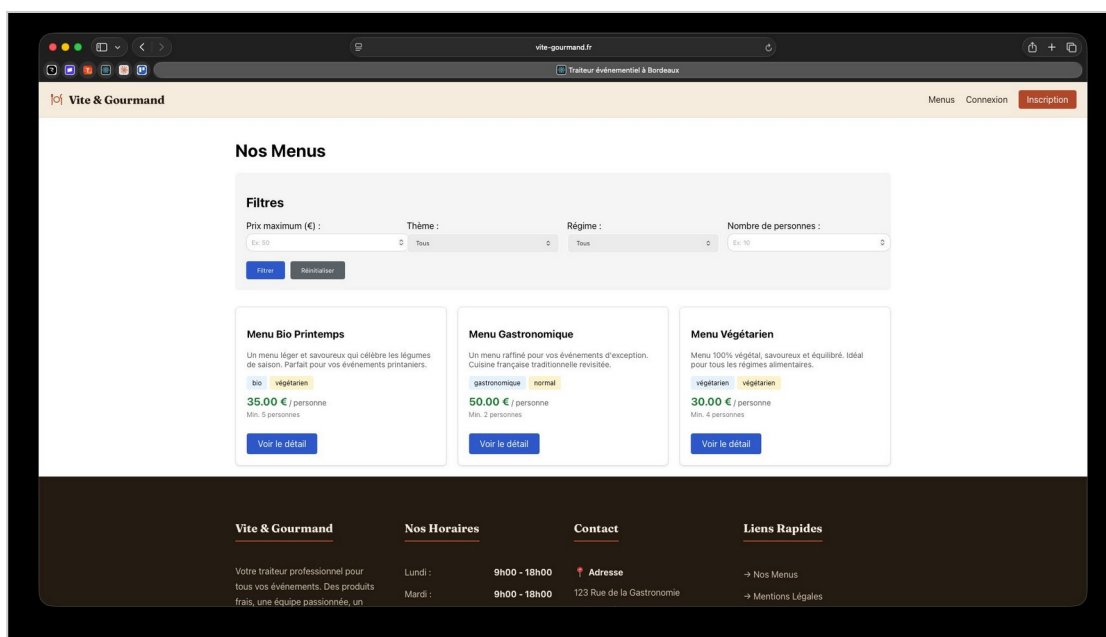


Figure A.2 – Liste des menus avec les filtres dynamiques (prix, thème, régime, nombre de personnes).

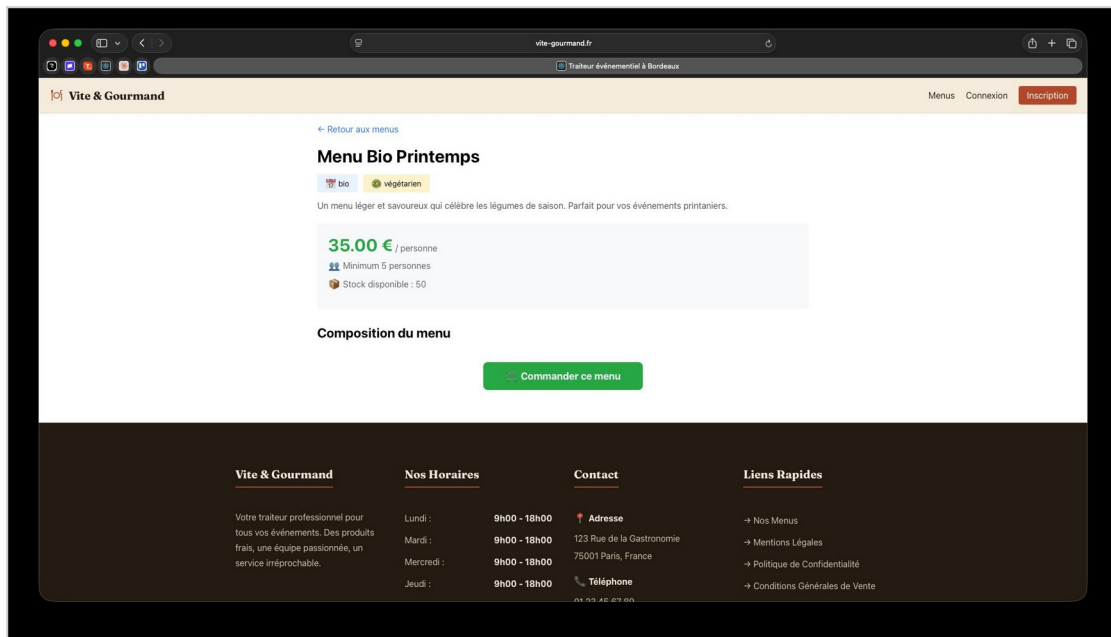


Figure A.3 – Fiche d'un menu : prix, minimum de personnes, stock disponible et composition.

Vite & Gourmand

Menus Mes Commandes Bonjour Demo Inscription

Commander : Menu Gastronomique

Prix : 50.00 € / personne

Minimum : 2 personnes

Nombre de personnes * :

Date de prestation * :

Heure de prestation * :

Adresse de livraison * :

Ville * : Code postal * :

☐ Prêt de matériel (vaisselle, couverts, nappes)

Distance de livraison (km) :

0 dans Bordeaux ; sinon 0,59 €/km en plus des 5 € de base.

[Valider la commande](#)

Figure A.4 – Formulaire de commande : calcul du prix, options de livraison et prêt de matériel.

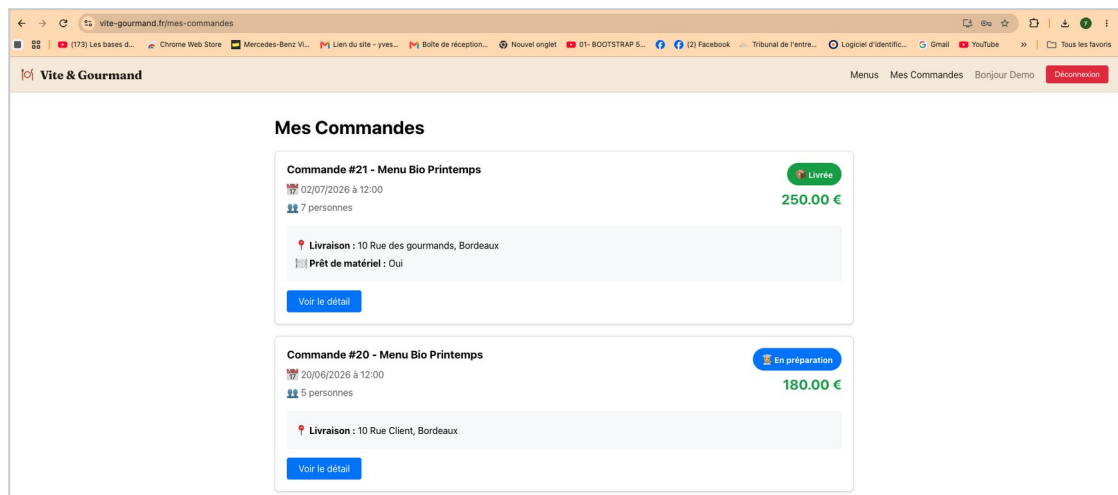


Figure A.5 – Espace client « Mes commandes » avec le suivi des statuts.

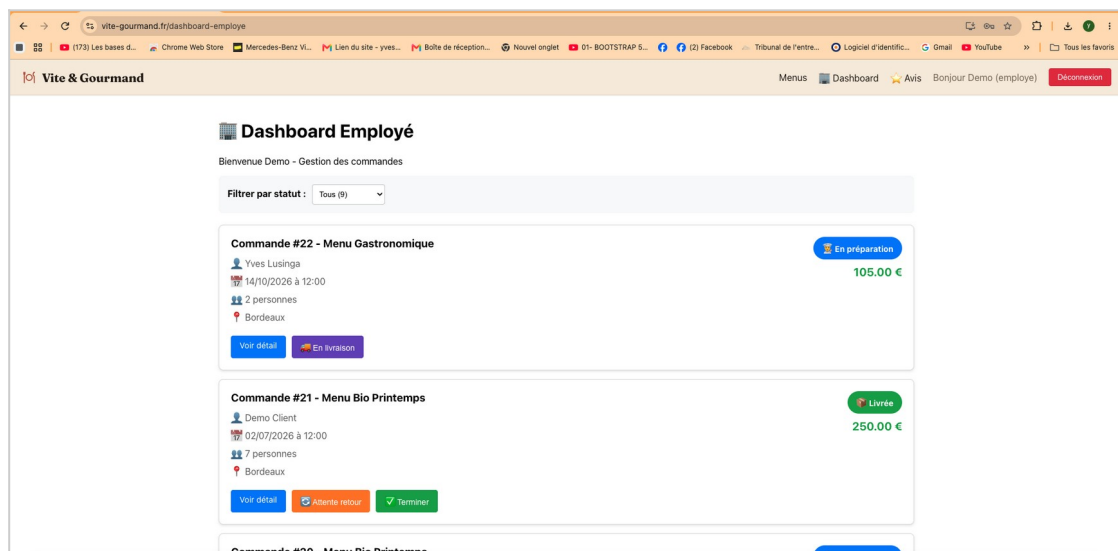


Figure A.6 – Tableau de bord employé : gestion et changement de statut des commandes.

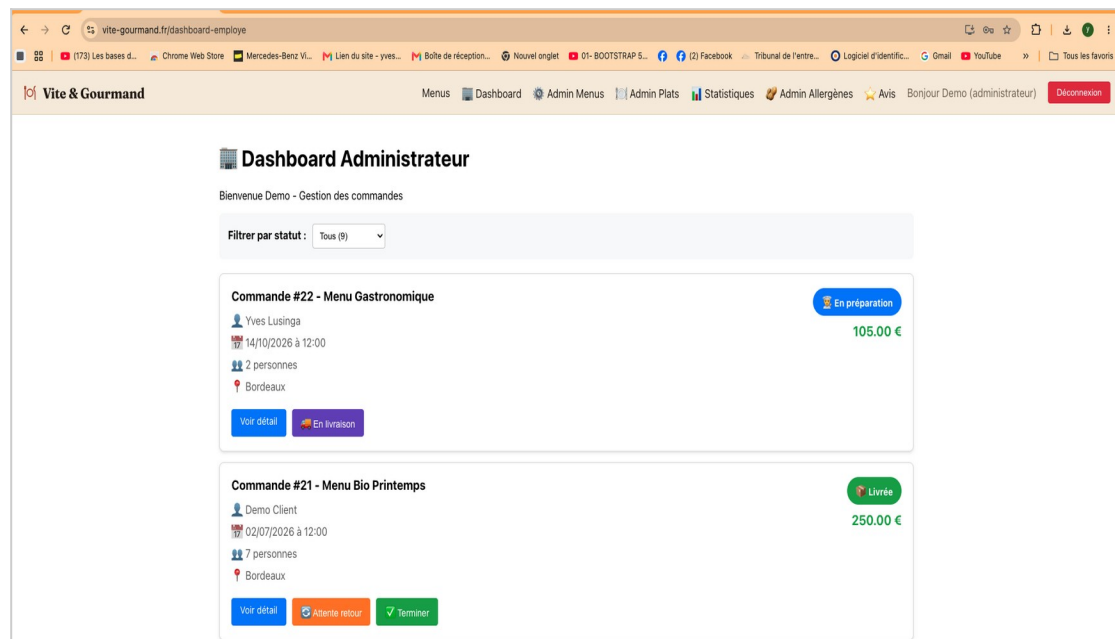


Figure A.7 – Tableau de bord administrateur avec accès aux espaces d'administration.

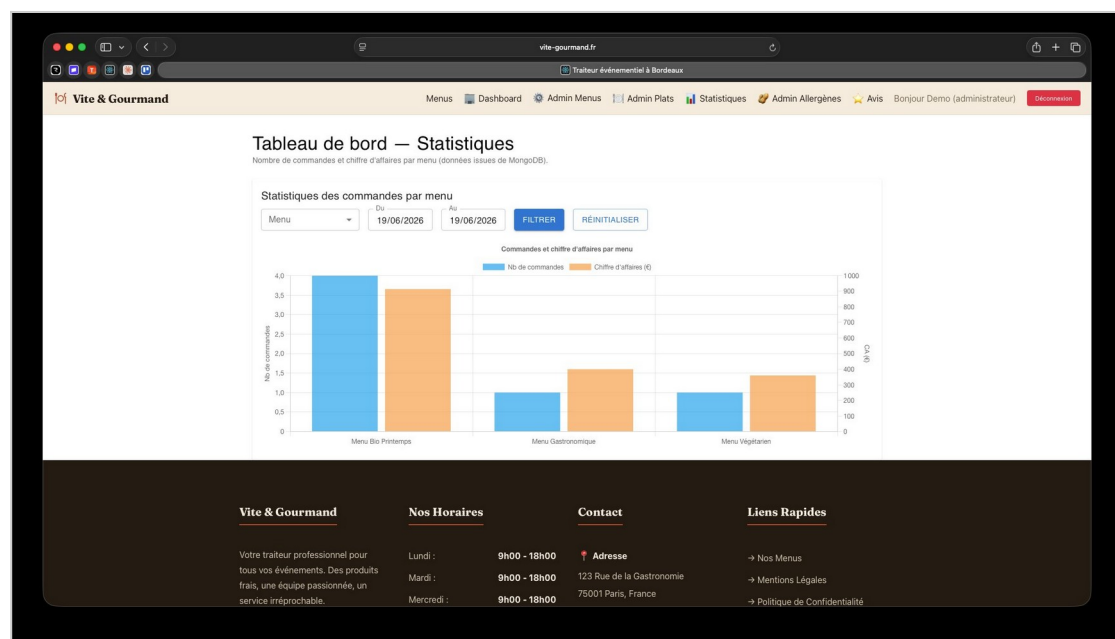


Figure A.8 – Statistiques des commandes et du chiffre d'affaires par menu (données MongoDB).

11 Difficultés rencontrées et solutions

11.1 Accès SSH au VPS

Problème : lors du déploiement, impossibilité de me connecter en SSH au VPS OVH (mot de passe systématiquement refusé, même après réinitialisation).

Solution : passage en mode rescue via l'interface OVH, montage du système de fichiers (mount /dev/sda1 /mnt), chroot /mnt, identification de l'utilisateur dev (grep

/bin/bash /etc/passwd), réinitialisation du mot de passe (passwd dev), redémarrage en mode normal, connexion SSH réussie.

Apprentissage : administration système Linux, gestion des utilisateurs et permissions, mode rescue et récupération système, débrouillardise face à un blocage technique.

11.2 Configuration de l'API frontend (CRA vs Vite)

Problème : après le build de production, le front continuait d'appeler `http://localhost:8000/api` au lieu de l'API de production.

Analyse : le projet utilise Create React App (préfixe `REACT_APP_`), et non Vite (`VITE_ / import.meta.env`). Le fichier `api.js` utilisait par erreur la syntaxe Vite.

Solution : création de `.env.production` avec `REACT_APP_API_URL=https://vite-gourmand.fr/api`, correction de `src/services/api.js` (`process.env.REACT_APP_API_URL`), rebuild complet (`rm -rf build && npm run build`).

Apprentissage : importance de bien comprendre l'outil de build utilisé et ses spécificités sur les variables d'environnement.

11.3 Routing Nginx pour l'API

Problème : les routes `/api/...` retournaient des erreurs 404 malgré une configuration en apparence correcte.

Solution finale :

```
location /api/ {
    root /var/www/vite-gourmand/backend/public;
    rewrite ^/api/(.*)$ /index.php?/$1 last;
}
location ~ \.php$ {
    root /var/www/vite-gourmand/backend/public;
    fastcgi_pass unix:/var/run/php/php8.2-fpm.sock;
    include fastcgi_params;
    fastcgi_param SCRIPT_FILENAME
    $document_root$fastcgi_script_name;
}
```

Apprentissage : directive `rewrite`, différence `root/alias`, fonctionnement de FastCGI, importance de l'ordre des blocs `location`.

11.4 Extension MongoDB en production

Problème : la mise en place des statistiques NoSQL nécessitait l'extension PHP MongoDB, absente du serveur.

Solution : installation de l'extension via `pecl install mongodb` puis activation dans la configuration PHP, et configuration de la connexion à MongoDB Atlas (URI + base de données) dans le `.env`.

Apprentissage : gestion des extensions PECL, intégration d'une base NoSQL infogérée à un projet Laravel.

11.5 Faille d'autorisation détectée par les tests

Problème : les tests automatisés ont révélé que les routes d'administration n'imposaient pas de contrôle de rôle : un utilisateur authentifié simple pouvait créer/modifier/supprimer des menus (réponse 201 au lieu de 403).

Solution : création d'un middleware `CheckRole` appliqué au groupe d'administration ; après correction, les 18 tests passent.

Apprentissage : importance des tests pour la sécurité et application concrète du principe de moindre privilège (broken access control – OWASP A01).

12 Bilan du projet

12.1 Objectifs atteints

- **Application complète et fonctionnelle** : frontend React responsive, backend Laravel robuste, bases PostgreSQL et MongoDB bien structurées.
- **Stack technique moderne** : technologies actuelles et recherchées, architecture découplée maintenable, code organisé et commenté.
- **Sécurité et conformité RGPD** : authentification par tokens, validation stricte, contrôle d'accès par rôle, protection contre les attaques courantes, droits RGPD implémentés.
- **Statistiques NoSQL et e-mails transactionnels** opérationnels.
- **Accessibilité RGAA** sur les pages principales.
- **Déployée en production** : application accessible sur <https://vite-gourmand.fr>, HTTPS valide, performance satisfaisante.
- **Testée et documentée** : 18 tests automatisés, documentation technique complète.

12.2 Compétences développées

12.2.1 Compétences techniques

Frontend (CCP 1) : interfaces React, gestion de l'état (Context API), routing (React Router), communication HTTP (Axios), accessibilité RGAA, responsive design.

Backend (CCP 2) : API REST Laravel, modélisation relationnelle (PostgreSQL) et documentaire (MongoDB), Eloquent ORM et requêtes optimisées, authentification et autorisation, validation et sécurité, tests automatisés.

Déploiement et sécurité (transversal aux deux CCP) : configuration serveur Linux (Ubuntu), Nginx, PostgreSQL, installation et configuration SSL, mise en production, gestion des secrets.

12.2.2 Compétences méthodologiques

- **Organisation** : planification et suivi des fonctionnalités (F01-F12)
- **Résolution de problèmes** : débrouillardise face aux blocages
- **Autonomie** : recherche de solutions et documentation
- **Rigueur** : tests, validation, sécurité
- **Documentation** : code commenté, README, dossier projet

12.3 Métriques du projet

- Développement initial : 12 jours, puis itérations d'amélioration
- 2 bases de données (PostgreSQL relationnel + MongoDB NoSQL)
- ~8 000 lignes de code (backend + frontend)
- Une dizaine de tables relationnelles + collection statistique NoSQL
- ~46 routes API REST
- **18 tests automatisés**
- 15+ pages / composants React

12.4 Perspectives d'évolution

12.4.1 Améliorations fonctionnelles

- **Païement en ligne** (Stripe ou PayPal)
- **Automatisation de la caution matériel** (tâche planifiée / cron déclenchant le rappel et la retenue après le délai de 10 jours)
- **Statistiques prédictives** (tendances de commandes, prévision de CA) exploitant la base NoSQL
- **Application mobile** (React Native) et **PWA**
- **Système de réservation** d'événements

12.4.2 Améliorations techniques

- **Cache Redis** pour les performances
- **Monitoring** centralisé (Sentry, métriques)
- **CI/CD** (automatisation des tests et déploiements)
- **Internationalisation** (i18n)
- Poursuite de la mise en accessibilité sur l'ensemble des formulaires

12.5 Bilan personnel

Ce projet m'a permis de mettre en pratique l'ensemble des compétences acquises durant ma formation. La détection d'une faille d'autorisation par les tests, l'intégration d'une base NoSQL et la mise en accessibilité ont particulièrement renforcé ma rigueur et ma compréhension des bonnes pratiques.

Points forts : maîtrise du stack complet (React + Laravel + PostgreSQL + MongoDB), capacité à déployer en production, résolution autonome de problèmes techniques, respect des bonnes pratiques de sécurité.

Points d'amélioration : élargir la couverture de tests, améliorer la gestion des erreurs frontend, poursuivre les optimisations (cache, lazy loading), renforcer la documentation du code.

13 Conclusion

Le projet Vite & Gourmand représente l'aboutissement de ma formation au Titre Professionnel Développeur Web et Web Mobile. J'ai conçu et développé une application full-stack moderne (React 19 + Laravel 10 + PostgreSQL + MongoDB), déployée en production sur un serveur sécurisé.

Réalisations principales :

- Application full-stack avec double persistance (relationnelle + NoSQL)
- 29 cas d'usage couvrant tous les besoins identifiés
- Règles métier avancées (statuts de commande, remise, frais de livraison, caution matériel)
- Sécurité robuste (HTTPS, Sanctum, validation, contrôle d'accès par rôle), faille corrigée grâce aux tests
- Conformité RGPD et accessibilité RGAA
- E-mails transactionnels (Resend) et statistiques NoSQL
- 18 tests automatisés
- Déploiement production avec SSL

Compétences validées — les 2 CCP du Titre DWWM :

- **CCP 1** : développer la partie front-end d'une application web sécurisée
- **CCP 2** : développer la partie back-end d'une application web sécurisée

Le déploiement et la sécurisation, transversaux, sont mis en œuvre tout au long du projet.

Cette application est *production-ready* et pourrait être utilisée par un véritable traiteur événementiel. L'architecture choisie permet d'évoluer facilement (ajout de fonctionnalités, montée en charge). Je suis désormais prêt à intégrer une équipe de développement, avec la volonté de continuer à apprendre et à me perfectionner.

14 Annexes

14.1 Annexe A : comptes de démonstration

- **Client** — Email : `client@demo.fr` — Mot de passe : `Password123!` — Rôle : utilisateur
- **Employé** — Email : `employe@demo.fr` — Mot de passe : `Password123!` — Rôle : employe
- **Administrateur** — Email : `admin@demo.fr` — Mot de passe : `Password123!` — Rôle : administrateur

 Vérifier que ces identifiants de démonstration sont toujours valides en production avant la soutenance.

14.2 Annexe B : liens utiles

- Application en production : <https://vite-gourmand.fr>
- Dépôt GitHub : <https://github.com/yvesnet9/vite-gourmand>
- Documentation technique : `README.md` du dépôt

14.3 Annexe C : technologies et versions

Frontend : React 19, React Router 6, Axios, Create React App 5

Backend : Laravel 10, PHP 8.2, Laravel Sanctum, Composer 2.x

Bases de données : PostgreSQL 14 (relationnel), MongoDB Atlas (NoSQL)

E-mails : Resend

Serveur : Ubuntu 22.04 LTS, Nginx 1.18.0, PHP-FPM 8.2, Node.js 18

Outils : Git, Certbot (Let's Encrypt), npm

14.4 Annexe D : glossaire

- **API** : Application Programming Interface — interface de communication entre le frontend et le backend
- **CRUD** : Create, Read, Update, Delete — opérations de base sur les données
- **MCD** : Modèle Conceptuel de Données
- **NoSQL** : base de données non relationnelle (ici orientée documents, MongoDB)
- **ORM** : Object-Relational Mapping (Eloquent)
- **RGAA** : Référentiel Général d'Amélioration de l'Accessibilité
- **RGPD** : Règlement Général sur la Protection des Données
- **REST** : Representational State Transfer — style d'architecture pour les API
- **SPA** : Single Page Application
- **SSL/TLS** : protocoles de sécurisation des échanges sur Internet
- **VPS** : Virtual Private Server

14.5 Annexe E : maquettes (wireframes)

Wireframes basse fidélité des écrans principaux, réalisés avant le développement pour valider la structure des pages et le parcours utilisateur (version bureau et mobile).

Maquettes — Vite & Gourmand

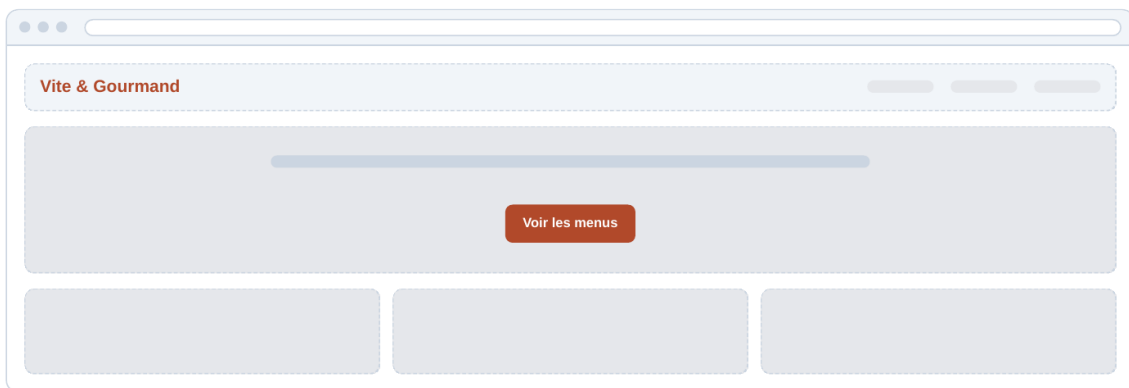
Wireframes basse-fidélité des écrans principaux · Application de traiteur événementiel

Jamesy MUKUNA MUKENKETAYI — Titre DWWM — Session Juin–Juillet 2026

Lecture : les blocs gris représentent des zones de contenu (textes, images), les cadres en pointillés des conteneurs, les rectangles verts les actions principales. Ces maquettes décrivent la **structure** des pages, pas le design final.

1 Page d'accueil

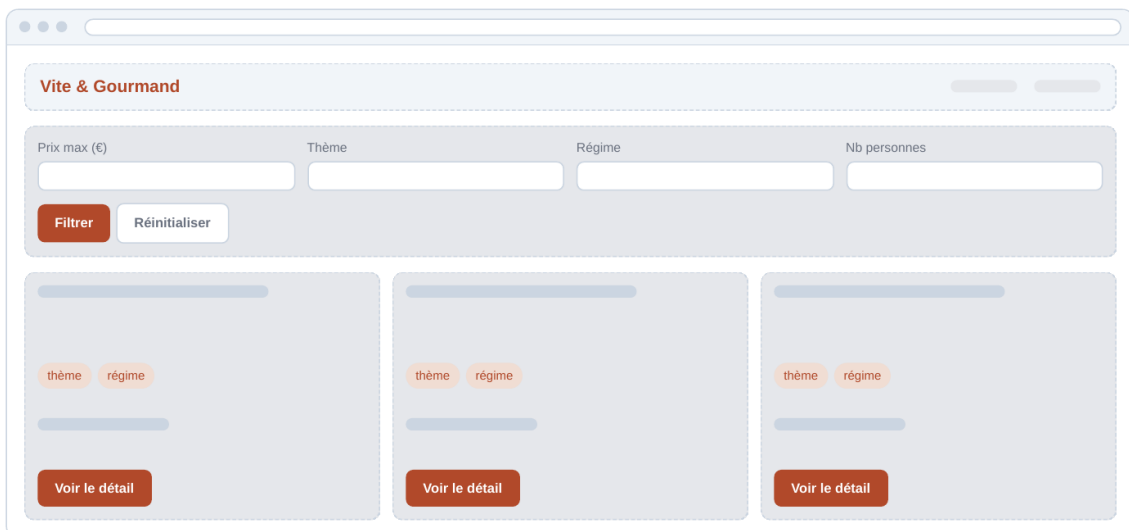
Point d'entrée public — présentation et accès au catalogue.



- Barre de navigation présente sur toutes les pages (logo + liens Menus / Connexion / Inscription).
- Bandeau d'accroche (slogan) avec appel à l'action vers le catalogue.

2 Liste des menus (avec filtres)

Catalogue filtrable par prix, thème, régime et nombre de personnes.



- Barre de filtres en haut ; chaque champ a un libellé (accessibilité).
- Cartes menu : titre, description, badges thème/régime, prix, bouton détail.

3 Détail d'un menu

Composition, allergènes, prix, avis et accès à la commande.

Vite & Gourmand

Allergène Allergène Allergène

Commander ce menu

- Visuel du menu + composition (plats) et liste des allergènes.
- Prix, note moyenne des avis validés, bouton « Commander ce menu ».

4 Connexion / Inscription

Authentification (Sanctum) et création de compte avec consentement RGPD.

Vite & Gourmand

Email

Mot de passe

Se connecter

Nom / Prénom

Email

Mot de passe

☐ J'accepte la politique de confidentialité

S'inscrire

- Chaque champ possède un libellé associé ; case RGPD obligatoire à l'inscription.
- Validation stricte du mot de passe (≥ 10 caractères, casse, chiffre, symbole).

5 Formulaire de commande

Date, adresse, quantité — prix calculé en temps réel (remise + livraison).

Vite & Gourmand

Menu sélectionné

Date de livraison (future)

Adresse de livraison

Quantité (≥ minimum du menu)

Valider la commande

- Récapitulatif : prix de base, remise 10 % (si quantité ≥ min + 5), frais de livraison (5 € + 0,59 €/km), total.
- Contrôles : date future, quantité ≥ minimum, stock suffisant.

6 Mes commandes (client)

Historique des commandes avec statuts et annulation possible.

Vite & Gourmand

Menu	Date livraison	Prix	Statut	Action
Menu Prestige	--/--	— €	en attente	Annuler
Menu Bio	--/--	— €	en préparation	—
Menu Mariage	--/--	— €	livrée	Laisser un avis

- Statuts colorés ; annulation visible uniquement si « en attente ».
- Invitation à laisser un avis lorsque la commande est terminée.

7 Dashboard employé

Suivi des commandes, gestion des statuts et modération des avis.

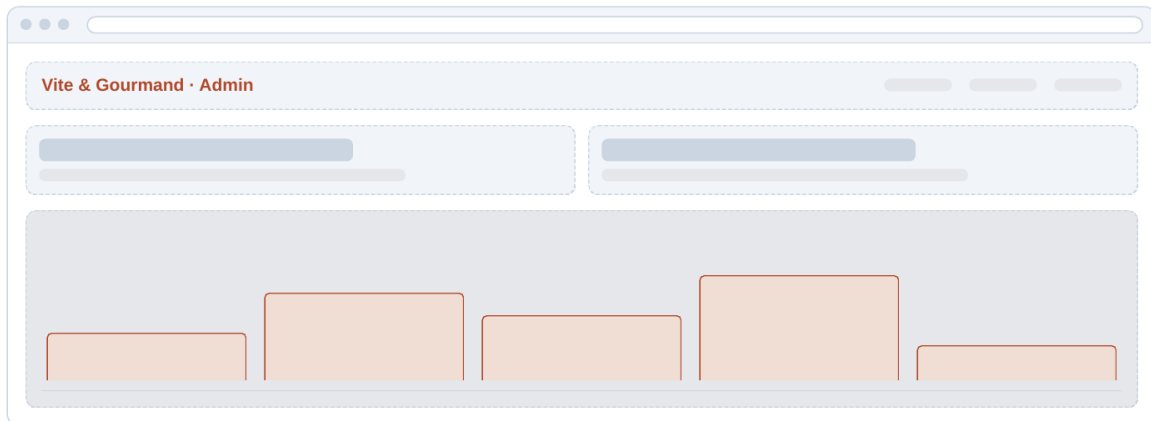
Vite & Gourmand · Employé

Commande	Client	Statut	Changer statut
#1024	—	en attente	→ acceptée
#1023	—	en préparation	→ en livraison

- Indicateurs : commandes en attente, avis à modérer, CA du jour.
- Évolution des statuts selon le workflow (8 statuts).

8 Dashboard administrateur — Statistiques (NoSQL)

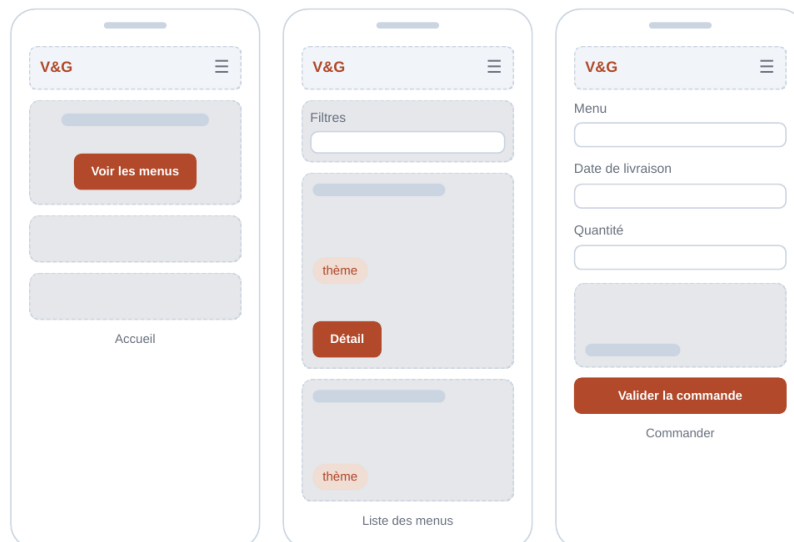
Graphique du chiffre d'affaires par menu, alimenté par MongoDB.



- Accès aux fonctions employé + gestion plats / allergènes / utilisateurs.
- Graphique CA par menu : données agrégées dans la collection NoSQL statistiques_commandes.

9 Versions mobile (responsive)

Le site est responsive : sur mobile, la navigation se replie en bouton « hamburger » et les contenus passent en une seule colonne.



- Navigation repliée en menu « hamburger » (avec `aria-label` et `aria-expanded` — accessibilité RGAA).
- Contenus empilés en une colonne, cibles tactiles agrandies, mise en page fluide jusqu'à ~360 px.

Document de conception — Vite & Gourmand · 8 écrans · à compléter par les captures de l'application en production (annexe du dossier).

14.6 Annexe F : charte graphique

Identité visuelle de Vite & Gourmand : logo, palette de couleurs, typographie, composants d'interface, espacements et accessibilité.

Charte graphique

Vite & Gourmand — Application de traiteur événementiel · Jamesy MUKUNA MUKENKETAYI — Titre DWWW — Session Juin–Juillet 2026

1. Logo

Un pictogramme « couvert + brin végétal » en terracotta, associé au nom en *Fraunces*. Le pictogramme peut s'utiliser seul (favicon, app).

 **Vite & Gourmand**

 **Vite & Gourmand**

2. Palette de couleurs

Une identité chaleureuse et artisanale : fond crème, texte brun, accent terracotta, touche verte « nature/bio ».

Crème #F5ECD0 Fond principal	Brun foncé #241B13 Texte principal	Terracotta #B1492A Accent, actions, logo	Beige #E3D5BF Bordures, séparateurs
Brun atténué #7D6F5C Texte secondaire	Vert nature #2C5F2D Secondaire (bio), thème	Blanc #FFFFFF Cartes, surfaces	

Couleurs d'état (messages & statuts) :

Succès · #166534	Information · #1E40AF	Alerte · #92400E	Erreur · #721C24
-----------------------------	----------------------------------	-----------------------------	-----------------------------

3. Typographie

Fraunces (serif, Google Fonts) pour les titres et le logo — caractère artisanal. Police **système sans-serif** pour le corps de texte — lisibilité optimale.

Display / H1 ·
Fraunces 700 · 32px

Vite & Gourmand

Titre / H2 ·
Fraunces 600 · 24px

Nos menus événementiels

Sous-titre / H3 ·
Fraunces 600 · 19px

Détail d'un menu

Corps · Système ·
16px

Consultez nos menus et commandez en quelques clics pour votre événement.

Petit / légende ·
13px

Prix indicatif, hors frais de livraison. Minimum de personnes selon le menu.

4. Composants d'interface

Boutons



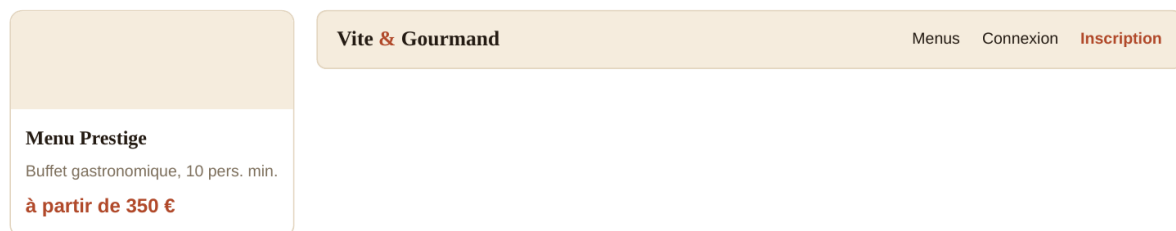
Champ de formulaire

Adresse de livraison

Badges de statut



Carte menu & barre de navigation



5. Espacements & rayons

Rayon boutons / champs — 4 à 6 px	Rayon cartes — 8 à 10 px
Échelle d'espacement — 4 / 8 / 12 / 16 / 24 px	Largeur de contenu — ~960 px centré

6. Accessibilité & ton

- **Contrastes** : combinaisons texte/fond visant un ratio $\geq 4,5:1$ (brun foncé sur crème, blanc sur terracotta) — cohérent avec le travail RGAA.
- **Focus visible** et libellés explicites sur tous les champs ; intitulés de liens compréhensibles hors contexte.
- **Ton** : convivial, artisanal et rassurant — on parle « menus », « événement », « livraison », pas de jargon technique.
- **Imagerie** : photos de plats chaleureuses, lumière naturelle, cadrage généreux ; le crème et le terracotta rappellent la cuisine et le fait-maison.

Charte graphique — Vite & Gourmand · identité relevée dans l'application en production.

FIN DU DOSSIER PROJET

Jamesy MUKUNA MUKENKETAYI — Session Novembre – Décembre 2026 — STUDI, Paris